

COMP 352 – Final Project: Footballer Market Evaluation

Goal: To predict the market value of a professional footballer next year

Authors: Zevin Attisha, Christian Vaikona, Santiago Guerrero, Santiago Pedetti

Dataset source: <https://www.kaggle.com/datasets/davidcariboo/player-scores>

Table of Contents

- Environment Setup
 - 1. Data Importing and Pre-processing
 - 2. Data Analysis and Visualization
 - 3. Data Analytics (Modeling)

Environment Setup

First we must setup our environment to make sure we have all appropriate modules installed. To do this, I have provided 2 methods:

METHOD 1

Install all modules using a `.yaml` file via `conda`.

To do this, run:

```
conda env create -f env_setup/environment.yml
```

Then activate the environment by:

```
conda activate data_env
```

METHOD 2

You can also use the `requirements.txt` file to download the modules via `pip`.

To do this, first make create and activate your environment:

```
conda create -n my_data_env
```

```
conda activate my_data_env
```

You may need to install setup tools. To do this run (Note you may need to change `pip3` to `pip`):

```
pip3 install --upgrade pip setuptools wheel
```

and then run:

```
pip3 install -r env_setup/requirements.txt
```

1. Data Importing and Pre-Processing

- Import dataset and describe characteristics such as dimensions, data types, file types, and import methods used
- Clean, wrangle, and handle missing data, duplicate data, etc.
- Encode any categorical variables
- Perform feature engineering on the dataset
- Transform data appropriately using techniques such as aggregation, normalization, and feature construction
- Reduce redundant data and perform need based discretization

We utilize four datasets:

- `player_valuations.csv` : multiple valuation records per player over time
- `players.csv` : one record per player with demographic and categorical info
- `appearances.csv` : one record per player with statistics per game appearance
- `game_lineups.csv` : one record per player with status per game

```
In [1]: # Importing libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, TimeSeriesSplit, cross_val_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression, ElasticNet
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Functions from other scripts trying to reduce repetition
from helper_scripts import (
    evaluate_model,
    cv_mae,
    remove_duplicates
)

# Dataframe formatting
pd.set_option("display.max_columns", None)
np.set_printoptions(suppress=True)
```

```
In [2]: # Creating variables for csv paths
PATH_PLAYER_VALUATIONS = "data/raw/player_valuations.csv"
PATH_PLAYERS = "data/raw/players.csv"
```

```

PATH_APPEARANCES = "data/raw/appearances.csv"
PATH_LINEUPS = "data/raw/game_lineups.csv"

# Creating Pandas dataframes
player_valuations_df = pd.read_csv(PATH_PLAYER_VALUATIONS)
players_df = pd.read_csv(PATH_PLAYERS)
appearances_df = pd.read_csv(PATH_APPEARANCES)
lineups_df = pd.read_csv(PATH_LINEUPS)

# Displaying dataframe shapes
print("-"*60)
print("Dataframe Shapes")
print("-"*60)

print("player_valuations_df shape:", player_valuations_df.shape)
print("players_df shape:", players_df.shape)
print("appearances_df shape:", appearances_df.shape)
print("lineups_df shape:", lineups_df.shape)
print("\n")

# Displaying dataframe heads
print("-"*60)
print("Dataframe Heads")
print("-"*60)

print("player_valuations_df:")
display(player_valuations_df.head())

print("\nplayers_df:")
display(players_df.head())

print("\nappearances_df:")
display(appearances_df.head())

print("\nlineups_df:")
display(lineups_df.head())

```

Dataframe Shapes

```

player_valuations_df shape: (496606, 5)
players_df shape: (32601, 23)
appearances_df shape: (1706806, 13)
lineups_df shape: (2191911, 10)

```

Dataframe Heads

```

player_valuations_df:

```

	player_id	date	market_value_in_eur	current_club_id	player_club_domestic_compe
0	405973	2000-01-20	150000	3057	
1	342216	2001-07-20	100000	1241	
2	3132	2003-12-09	400000	126	
3	6893	2003-12-15	900000	984	
4	10	2004-10-04	7000000	398	

players_df:

	player_id	first_name	last_name	name	last_season	current_club_id	player.
0	10	Miroslav	Klose	Miroslav Klose	2015	398	mir
1	26	Roman	Weidenfeller	Roman Weidenfeller	2017	16	ro weide
2	65	Dimitar	Berbatov	Dimitar Berbatov	2015	1091	di ber
3	77	NaN	Lúcio	Lúcio	2012	506	
4	80	Tom	Starke	Tom Starke	2017	27	tom-

appearances_df:

	appearance_id	game_id	player_id	player_club_id	player_current_club_id	date	p
0	2231978_38004	2231978	38004	853	235	2012-07-03	
1	2233748_79232	2233748	79232	8841	2698	2012-07-05	
2	2234413_42792	2234413	42792	6251	465	2012-07-05	
3	2234418_73333	2234418	73333	1274	6646	2012-07-05	
4	2234421_122011	2234421	122011	195	3008	2012-07-05	

lineups_df:

		game_lineups_id	date	game_id	player_id	club_id	player_name
0	b2dbe01c3656b06c8e23e9de714e26bb		2013-07-27	2317258	1443	610	Christian Poulsen
1	b50a3ec6d52fd1490aab42042ac4f738		2013-07-27	2317258	5017	610	Niklas Moisander
2	7d890e6d0ff8af84b065839966a0ec81		2013-07-27	2317258	9602	1090	Maarten Martens
3	8c355268678b9bbc7084221b1f0fde36		2013-07-27	2317258	12282	610	Daley Blind
4	76193074d549e5fdce4cdcbba0d66247		2013-07-27	2317258	25427	1090	Roy Beerens

In [3]: *# Checking for duplicates*

```
remove_duplicates("Player Valuations Dataset", player_valuations_df)
remove_duplicates("Players Dataset", players_df)
remove_duplicates("Appearances Dataset", appearances_df)
remove_duplicates("Lineups Dataset", lineups_df)
```

Player Valuations Dataset

Total rows: 496606
Duplicate rows: 0
No duplicate rows found.

Players Dataset

Total rows: 32601
Duplicate rows: 0
No duplicate rows found.

Appearances Dataset

Total rows: 1706806
Duplicate rows: 0
No duplicate rows found.

Lineups Dataset

Total rows: 2191911
Duplicate rows: 0
No duplicate rows found.

```

In [4]: # Types per dataset
print("player_valuations_df dtypes:")
display(player_valuations_df.dtypes)

print("\nplayers_df dtypes:")
display(players_df.dtypes)

print("\nappearances_df dtypes:")
display(appearances_df.dtypes)

print("\nlineups_df dtypes:")
display(lineups_df.dtypes)
print("\n")

# Columns that display the most missing values in descending order
print("-"*60)
print("Missing Values in player_valuations_df:")
print("-"*60)
display(player_valuations_df.isna().sum().sort_values(ascending=False).head(10))

print("-"*60)
print("Missing values in players_df:")
print("-"*60)
display(players_df.isna().sum().sort_values(ascending=False).head(len(players_df)))

print("-"*60)
print("Missing values in appearances_df:")
print("-"*60)
display(appearances_df.isna().sum().sort_values(ascending=False).head(len(appearances_df)))

print("-"*60)
print("Missing values in lineups_df:")
print("-"*60)
display(lineups_df.isna().sum().sort_values(ascending=False).head(len(lineups_df)))

player_valuations_df dtypes:
player_id                int64
date                    object
market_value_in_eur      int64
current_club_id          int64
player_club_domestic_competition_id  object
dtype: object
players_df dtypes:

```

```

player_id          int64
first_name         object
last_name          object
name              object
last_season        int64
current_club_id    int64
player_code        object
country_of_birth   object
city_of_birth      object
country_of_citizenship object
date_of_birth      object
sub_position       object
position           object
foot               object
height_in_cm       float64
contract_expiration_date object
agent_name         object
image_url          object
url                object
current_club_domestic_competition_id object
current_club_name  object
market_value_in_eur float64
highest_market_value_in_eur float64
dtype: object
appearances_df dtypes:
appearance_id      object
game_id            int64
player_id          int64
player_club_id     int64
player_current_club_id int64
date               object
player_name        object
competition_id     object
yellow_cards       int64
red_cards          int64
goals              int64
assists            int64
minutes_played     int64
dtype: object
lineups_df dtypes:
game_lineups_id    object
date               object
game_id            int64
player_id          int64
club_id            int64
player_name        object
type               object
position           object
number             object
team_captain       int64
dtype: object

```

Missing Values in player_valuations_df:

```
player_id          0
date               0
market_value_in_eur 0
current_club_id    0
player_club_domestic_competition_id 0
dtype: int64
```

Missing values in players_df:

```
agent_name          16019
contract_expiration_date 12091
country_of_birth    2799
foot                2536
city_of_birth       2455
height_in_cm        2256
first_name          2062
highest_market_value_in_eur 1523
market_value_in_eur 1523
country_of_citizenship 383
sub_position        180
date_of_birth       47
position            0
player_code         0
current_club_id     0
last_season         0
name                0
image_url           0
url                 0
current_club_domestic_competition_id 0
current_club_name   0
last_name           0
player_id           0
dtype: int64
```

Missing values in appearances_df:

```
player_name        6
appearance_id       0
game_id            0
player_id          0
player_club_id     0
player_current_club_id 0
date               0
competition_id     0
yellow_cards       0
red_cards          0
goals              0
assists            0
minutes_played     0
dtype: int64
```

Missing values in lineups_df:

```

position          3
game_lineups_id   0
date              0
game_id           0
player_id         0
club_id           0
player_name       0
type              0
number            0
team_captain      0
dtype: int64

```

```

In [5]: # Clean players_df

# Copy to avoid altering original data
p = players_df.copy()

# Sort by player
p = p.sort_values(['player_id']).reset_index(drop=True)

# Drop where target variable isn't available
p = p.dropna(subset=["highest_market_value_in_eur"]) # Also ends up clearing

# Parse birth date
p["date_of_birth"] = pd.to_datetime(p["date_of_birth"], errors="coerce")

# Birth date required for age feature
p = p.dropna(subset=["date_of_birth"])

# Having both position and sub_position provides cleaner statistical analysis
p = p.dropna(subset=["sub_position"]) # Removes "Missing" values from position

# Drop to make right vs left foot correlation feature or impute for general
p = p.dropna(subset=["foot"])
# OR
# p["foot"] = p["foot"].fillna("Missing")

# Save original names before encoding
p['sub_position_name'] = p['sub_position'].copy()
p['position_name'] = p['position'].copy()
p['foot_name'] = p['foot'].copy()

# Label encoding for foot
foot_map = {'left': 0, 'right': 1, 'both': 2}
p['foot_encoded'] = p['foot'].map(foot_map)

# One-hot encoding for position
p = pd.get_dummies(p, columns=['position'], prefix='pos', drop_first=False,
position_cols = [col for col in p.columns if col.startswith('pos_')])

# Frequency encoding for sub_position
subpos_freq = p['sub_position'].value_counts(normalize=True)
p['sub_position'] = p['sub_position'].map(subpos_freq)
p = p.rename(columns={'sub_position': 'sub_position_freq'})

# Handle missing categoricals

```

```

# p["sub_position"] = p["sub_position"].fillna("Missing")

# # Numeric height + impute (less than 3% of height values were null)
p["height_in_cm"] = pd.to_numeric(p["height_in_cm"], errors="coerce")
p["height_in_cm"] = p["height_in_cm"].fillna(p["height_in_cm"].median())

# Keep only safe player-level features (no bias/leakage)
"""
Dropping The Following:

[Cannot Impute]
agent_name
contract_expiration_date
first_name
country_of_birth
city_of_birth
country_of_citizenship

[Irrelevant/Bias]
market_value_in_eur
current_club_domestic_competition_id
url
image_url
player_code
last_season
name
last_name
highest_market_value_in_eur

[Encoded]
foot - label encoding - simpler 0/1/2 to left/right/both
position - one-hot encoding - only 4 series added
sub_position - frequency encoding - At risk of data leakage if 13 different

[Potentially Bring Back For Feature]
current_club_name OR current_club_id - frequency of players at a club correlated
"""
keep_player_cols = [
    "player_id",
    "height_in_cm",
    "sub_position_name",
    "sub_position_freq",
    "foot_name",
    "foot_encoded",
    "date_of_birth",
    "position_name"
] + position_cols
p = p[keep_player_cols].copy()

print("p shape:", p.shape)
p.head()

# Keep here at bottom to check on df when needed
# print("--*60)
# print("Missing values in players_df:")

```

```
# print("-"*60)
# display(p.isna().sum().sort_values(ascending=False).head(len(p)))
```

p shape: (29054, 12)

```
Out [5]:
```

	player_id	height_in_cm	sub_position_name	sub_position_freq	foot_name	foot_e
0	10	184.0	Centre-Forward	0.140497	right	
1	26	190.0	Goalkeeper	0.104977	left	
4	80	194.0	Goalkeeper	0.104977	right	
7	132	179.0	Attacking Midfield	0.068803	both	
8	162	193.0	Goalkeeper	0.104977	right	

```
In [6]: # Clean appearances_df

# Copy to avoid altering original data
app = appearances_df.copy()

# Parse date to datetime
app['date'] = pd.to_datetime(app['date'], errors='coerce')

# Extract year from match date
app['year'] = app['date'].dt.year

# Fill missing performance stats with 0 (player appeared but didn't score/as
app['yellow_cards'] = app['yellow_cards'].fillna(0)
app['red_cards'] = app['red_cards'].fillna(0)
app['goals'] = app['goals'].fillna(0)
app['assists'] = app['assists'].fillna(0)
app['minutes_played'] = app['minutes_played'].fillna(0)

# Keep only safe player-level features (no bias/leakage)
"""
Dropping The Following:

player_name – player_id is present
appearance_id
game_id
player_club_id
player_current_club_id
competition_id
"""
keep_player_cols = [
    "player_id",
    "date",
    "yellow_cards",
    "red_cards",
    "goals",
    "assists",
    "minutes_played",
    "year"
]
app = app[keep_player_cols].copy()
```

```

print("app shape:", app.shape)
app.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in appearances_df:")
# print("-"*60)
# display(app.isna().sum().sort_values(ascending=False).head(len(app)))

```

app shape: (1706806, 8)

Out [6]:

	player_id	date	yellow_cards	red_cards	goals	assists	minutes_played	year
0	38004	2012-07-03	0	0	2	0	90	2012
1	79232	2012-07-05	0	0	0	0	90	2012
2	42792	2012-07-05	0	0	0	0	45	2012
3	73333	2012-07-05	0	0	0	0	90	2012
4	122011	2012-07-05	0	0	0	1	90	2012

In [7]:

```

# Clean player_valuations_df

# Copy to avoid altering original data
pv = player_valuations_df.copy()

# Parse date
pv["date"] = pd.to_datetime(pv["date"], errors="coerce")

# Sort by player and date
pv = pv.sort_values(['player_id', 'date']).reset_index(drop=True)

"""
Dropping the following:

current_club_id
player_club_domestic_competition_id
"""

keep_player_cols = [
    "player_id",
    "date",
    "market_value_in_eur"
]
pv = pv[keep_player_cols].copy()

```

```

print(f"pv shape: {pv.shape}")
print(f"Date range: {pv['date'].min()} to {pv['date'].max()}")
print(f"Unique players: {pv['player_id'].nunique()}")
print("pv head:")
pv.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in player_valuations_df:")
# print("-"*60)
# display(pv.isna().sum().sort_values(ascending=False).head(len(pv)))

```

pv shape: (496606, 3)
 Date range: 2000-01-20 00:00:00 to 2025-04-06 00:00:00
 Unique players: 31078
 pv head:

Out [7]:

	player_id	date	market_value_in_eur
0	10	2004-10-04	7000000
1	10	2005-01-07	9000000
2	10	2005-05-05	12000000
3	10	2005-09-30	15000000
4	10	2006-01-09	20000000

In [8]:

```

# Clean lineups_df

# Copy to avoid altering original data
lu = lineups_df.copy()

# Parse date
lu["date"] = pd.to_datetime(lu["date"], errors="coerce")

# Sort by player and date
lu = lu.sort_values(['player_id', 'date']).reset_index(drop=True)

# Dropping where position is not available
# lu = lu.dropna(subset=["position"])

# Renaming type column and label encoding values
# Simplified to indicate if starter or not per appearance
lu = lu.rename(columns={"type": "is_starter"})
type_map = {'substitutes': 0, 'starting_lineup': 1}
lu['is_starter'] = lu['is_starter'].map(type_map)

# Dropping where position is outdated or categorical, not specific
# lu = lu[~lu['position'].isin(['midfield', 'Attack', 'Defender', 'Sweeper'])]

"""
Dropping the following:

game_lineups_id
game_id

```

```

club_id
player_name
number
position - present in p (cleaned players_df)
"""
keep_player_cols = [
    "date",
    "player_id",
    "is_starter",
    "team_captain"
]
lu = lu[keep_player_cols].copy()
print("lu shape:", lu.shape)
lu.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in lineups_df:")
# print("-"*60)
# display(lu.isna().sum().sort_values(ascending=False).head(len(lu)))

```

lu shape: (2191911, 4)

Out [8]:

	date	player_id	is_starter	team_captain
0	2013-08-18	10	1	0
1	2013-08-25	10	1	0
2	2013-08-31	10	1	0
3	2013-09-15	10	1	0
4	2013-09-19	10	0	0

In [9]:

```

# Lifetime Cumulative Statistics - app

# Sort by player and date
app = app.sort_values(['player_id', 'date']).reset_index(drop=True)

# Creating cumulative features
app['total_goals'] = app.groupby(['player_id'])['goals'].cumsum()
app['total_assists'] = app.groupby(['player_id'])['assists'].cumsum()
app['total_minutes'] = app.groupby(['player_id'])['minutes_played'].cumsum()
app['total_yellow_cards'] = app.groupby(['player_id'])['yellow_cards'].cumsum()
app['total_red_cards'] = app.groupby(['player_id'])['red_cards'].cumsum()
app['total_appearances'] = app.groupby(['player_id']).cumcount() + 1 # +1 because first row is 0

'''grit feature- FEATURE FOR GOAL to yellow or red card CORRELATION

...

# Goal, assist, yellow-card, red-card rate
app['goal_rate'] = round(app['total_goals'] / app['total_appearances'],3)
app['assist_rate'] = round(app['total_assists'] / app['total_appearances'],3)
app['yellow_card_rate'] = round(app['total_yellow_cards'] / app['total_appearances'],3)
app['red_card_rate'] = round(app['total_red_cards'] / app['total_appearances'],3)

```

```

# Keep only the most recent row per player per date (in case multiple games
app_cumulative = app.groupby(['player_id', 'date']).tail(1).reset_index(drop

# Select columns for merge
app_for_merge = app_cumulative[[
    'player_id', 'date',
    'total_goals', 'total_assists', 'total_minutes',
    'total_yellow_cards', 'total_red_cards', 'total_appearances',
    'goal_rate', 'assist_rate', 'yellow_card_rate', 'red_card_rate'
]].copy()

# Verify cumulative logic is working correctly
print("\nChecking cumulative growth for a sample player:")
sample_player_id = app_for_merge['player_id'].iloc[100] if len(app_for_merge
sample_data = app_for_merge[app_for_merge['player_id'] == sample_player_id].
print(f"\nPlayer ID: {sample_player_id}")
display(sample_data[['date', 'total_goals', 'total_assists', 'total_minutes',
                    'goal_rate', 'assist_rate', 'yellow_card_rate', 'red_ca
print("All numerical stats should either remain constant or increase as time

print(f"\napp_for_merge shape: {app_for_merge.shape}")
print(f"Date range: {app_for_merge['date'].min()} to {app_for_merge['date'].
print(f"Unique players: {app_for_merge['player_id'].nunique()}")
print("\napp_for_merge head:")
app_for_merge.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in app_for_merge:")
# print("-"*60)
# display(app_for_merge.isna().sum().sort_values(ascending=False).head(len(a

```

Checking cumulative growth for a sample player:

Player ID: 10

	date	total_goals	total_assists	total_minutes	total_yellow_cards	total_red_cards
0	2012-08-23	1	0	83	1	0
1	2012-08-26	1	0	173	2	0
2	2012-09-02	3	0	262	3	0
3	2012-09-16	4	0	352	3	0
4	2012-09-20	4	0	442	3	0
5	2012-09-23	4	0	456	3	0
6	2012-09-26	4	0	546	3	0
7	2012-09-30	4	1	636	4	0
8	2012-10-07	6	1	708	4	0
9	2012-10-20	7	1	798	4	0

All numerical stats should either remain constant or increase as time goes on

app_for_merge shape: (1706792, 12)

Date range: 2012-07-03 00:00:00 to 2025-04-10 00:00:00

Unique players: 25689

app_for_merge head:

Out[9]:

	player_id	date	total_goals	total_assists	total_minutes	total_yellow_cards	total
--	-----------	------	-------------	---------------	---------------	--------------------	-------

0	10	2012-08-23	1	0	83	1	
1	10	2012-08-26	1	0	173	2	
2	10	2012-09-02	3	0	262	3	
3	10	2012-09-16	4	0	352	3	
4	10	2012-09-20	4	0	442	3	

```
In [10]: # Lifetime Cumulative Statistics - lu

# Sort by player and date
lu = lu.sort_values(['player_id', 'date']).reset_index(drop=True)

# Creating cumulative features
lu['total_starts'] = lu.groupby(['player_id'])['is_starter'].cumsum()
lu['total_captaincies'] = lu.groupby(['player_id'])['team_captain'].cumsum()

# Create a total games count within the lu dataframe itself
lu['total_games_in_lineup'] = lu.groupby(['player_id']).cumcount() + 1

# Live rates for tendency to start or be captain throughout career
lu['captain_rate'] = round(lu['total_captaincies'] / lu['total_games_in_lineup'])
lu['starting_rate'] = round(lu['total_starts'] / lu['total_games_in_lineup'])

# Keep only the most recent row per player per date (in case multiple games)
lu_cumulative = lu.groupby(['player_id', 'date']).tail(1).reset_index(drop=True)

# Select columns for merge
lu_for_merge = lu_cumulative[['player_id',
                                'date',
                                'is_starter',
                                'total_starts',
                                'team_captain',
                                'total_captaincies',
                                'starting_rate',
                                'captain_rate']]

# Verify cumulative logic is working correctly
print("\nChecking cumulative growth for a sample player:")
sample_player_id = lu_for_merge['player_id'].iloc[100] if len(lu_for_merge) > 100 else lu_for_merge['player_id'].iloc[0]
sample_data = lu_for_merge[lu_for_merge['player_id'] == sample_player_id].he
```

```

print(f"\nPlayer ID: {sample_player_id}")
display(sample_data[['date', 'total_starts', 'total_captaincies', 'starting_
print("All stats should either remain constant or increase as time goes on")

print(f"\nlu_for_merge shape: {lu_for_merge.shape}")
print(f"Date range: {lu_for_merge['date'].min()} to {lu_for_merge['date'].ma
print(f"Unique players: {lu_for_merge['player_id'].nunique()}")
print("\nlu_for_merge head:")
lu_for_merge.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in lu_for_merge:")
# print("-"*60)
# display(lu_for_merge.isna().sum().sort_values(ascending=False).head(len(lu

```

Checking cumulative growth for a sample player:

Player ID: 10

	date	total_starts	total_captaincies	starting_rate	captain_rate
0	2013-08-18	1	0	1.000	0.0
1	2013-08-25	2	0	1.000	0.0
2	2013-08-31	3	0	1.000	0.0
3	2013-09-15	4	0	1.000	0.0
4	2013-09-19	4	0	0.800	0.0
5	2013-09-22	5	0	0.833	0.0
6	2013-10-20	5	0	0.714	0.0
7	2013-10-27	5	0	0.625	0.0
8	2013-10-30	6	0	0.667	0.0
9	2013-11-03	7	0	0.700	0.0

All stats should either remain constant or increase as time goes on

lu_for_merge shape: (2191895, 8)

Date range: 2013-07-02 00:00:00 to 2024-03-17 00:00:00

Unique players: 85235

lu_for_merge head:

Out[10]:

	player_id	date	is_starter	total_starts	team_captain	total_captaincies	starting_
0	10	2013-08-18	1	1	0	0	
1	10	2013-08-25	1	2	0	0	
2	10	2013-08-31	1	3	0	0	
3	10	2013-09-15	1	4	0	0	
4	10	2013-09-19	0	4	0	0	

CHECKPOINT 1

4 Tables have been curated:

- `p` : player_id, height_in_cm, sub_position, foot_encoded, date_of_birth, pos_Attack, pos_Defender, pos_Goalkeeper, pos_Midfield
- `pv` : player_id, date, market_value_in_eur
- `app_for_merge` : player_id, date, total_goals, total_assists, total_minutes, total_yellow_cards, total_red_cards, total_appearances, goal_rate, assist_rate, yellow_card_rate, red_card_rate
- `lu_for_merge` : player_id, date, is_starter, total_starts, team_captain, total_captaincies, starting_rate, captain_rate

```
In [11]: print(f"p shape: {p.shape}")
print(f"Unique players: {p['player_id'].nunique()}")
print("p head:")
display(p.head())

print(f"pv shape: {pv.shape}")
print(f"Date range: {pv['date'].min()} to {pv['date'].max()}")
print(f"Unique players: {pv['player_id'].nunique()}")
print("pv head:")
display(pv.head())

print(f"app_for_merge shape: {app_for_merge.shape}")
print(f"Date range: {app_for_merge['date'].min()} to {app_for_merge['date'].max()}")
print(f"Unique players: {app_for_merge['player_id'].nunique()}")
print("app_for_merge head:")
display(app_for_merge.head())

print(f"lu_for_merge shape: {lu_for_merge.shape}")
```

```

print(f"Date range: {lu_for_merge['date'].min()} to {lu_for_merge['date'].max()}")
print(f"Unique players: {lu_for_merge['player_id'].nunique()}")
print("lu_for_merge head:")
display(lu_for_merge.head())

```

p shape: (29054, 12)
 Unique players: 29054
 p head:

	player_id	height_in_cm	sub_position_name	sub_position_freq	foot_name	foot_end
0	10	184.0	Centre-Forward	0.140497	right	
1	26	190.0	Goalkeeper	0.104977	left	
4	80	194.0	Goalkeeper	0.104977	right	
7	132	179.0	Attacking Midfield	0.068803	both	
8	162	193.0	Goalkeeper	0.104977	right	

pv shape: (496606, 3)
 Date range: 2000-01-20 00:00:00 to 2025-04-06 00:00:00
 Unique players: 31078
 pv head:

	player_id	date	market_value_in_eur
0	10	2004-10-04	7000000
1	10	2005-01-07	9000000
2	10	2005-05-05	12000000
3	10	2005-09-30	15000000
4	10	2006-01-09	20000000

app_for_merge shape: (1706792, 12)
 Date range: 2012-07-03 00:00:00 to 2025-04-10 00:00:00
 Unique players: 25689
 app_for_merge head:

	player_id	date	total_goals	total_assists	total_minutes	total_yellow_cards	total_r
0	10	2012-08-23	1	0	83	1	
1	10	2012-08-26	1	0	173	2	
2	10	2012-09-02	3	0	262	3	
3	10	2012-09-16	4	0	352	3	
4	10	2012-09-20	4	0	442	3	

lu_for_merge shape: (2191895, 8)
 Date range: 2013-07-02 00:00:00 to 2024-03-17 00:00:00
 Unique players: 85235
 lu_for_merge head:

	player_id	date	is_starter	total_starts	team_captain	total_captaincies	starting_r
0	10	2013-08-18	1	1	0	0	
1	10	2013-08-25	1	2	0	0	
2	10	2013-08-31	1	3	0	0	
3	10	2013-09-15	1	4	0	0	
4	10	2013-09-19	0	4	0	0	

```
In [12]: # Merging pv and p

# Merge pv (Player Valuations) with p (Player Info)
valuation_profiles = pv.merge(p, on='player_id', how='left')

# Choosing series to drop all rows with null values at all
# Rows only have nulls if there was no player information to join to the val
# Dropped records should account for all records of any player_id that was c
valuation_profiles = valuation_profiles.dropna(subset=["foot_name"])
```

```

# Create age at valuation
valuation_profiles['age_at_valuation'] = round((valuation_profiles['date'] -

# Ensuring general types
valuation_profiles = valuation_profiles.astype({'player_id': int,
        'market_value_in_eur': int,
        'height_in_cm': float,
        'sub_position_name': str,
        'sub_position_freq': float,
        'foot_name': str,
        'foot_encoded': int,
        'position_name': str,
        'pos_Attack': int,
        'pos_Defender': int,
        'pos_Goalkeeper': int,
        'pos_Midfield': int,
        'age_at_valuation': float
    })

# Ensuring date types
valuation_profiles['date'] = pd.to_datetime(valuation_profiles['date'])
valuation_profiles['date_of_birth'] = pd.to_datetime(valuation_profiles['date_of_birth'])

# Ensuring data is sorted
valuation_profiles = valuation_profiles.sort_values(['player_id', 'date']).reset_index(drop=True)

print("valuation_profiles shape:", valuation_profiles.shape)
print(f"Date range: {valuation_profiles['date'].min()} to {valuation_profiles['date'].max()}")
valuation_profiles.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in valuation_profiles:")
# print("-"*60)
# display(valuation_profiles.isna().sum().sort_values(ascending=False).head(10))

```

valuation_profiles shape: (480533, 15)

Date range: 2001-07-20 00:00:00 to 2025-04-06 00:00:00

Out[12]:

	player_id	date	market_value_in_eur	height_in_cm	sub_position_name	sub_pos
--	-----------	------	---------------------	--------------	-------------------	---------

0	10	2004-10-04	7000000	184.0	Centre-Forward	
---	----	------------	---------	-------	----------------	--

1	10	2005-01-07	9000000	184.0	Centre-Forward	
---	----	------------	---------	-------	----------------	--

2	10	2005-05-05	12000000	184.0	Centre-Forward	
---	----	------------	----------	-------	----------------	--

3	10	2005-09-30	15000000	184.0	Centre-Forward	
---	----	------------	----------	-------	----------------	--

4	10	2006-01-09	20000000	184.0	Centre-Forward	
---	----	------------	----------	-------	----------------	--

```

In [13]: # Merging app_for_merge with lu_for_merge

# Merging app_for_merge (Appearance Info) with lu_for_merge (Lineup Info)
game_stats = app_for_merge.merge(lu_for_merge, on=['player_id', 'date'], how='outer')

# Time discrepancy from initial date between both datasets
game_stats.drop(columns=['total_starts', 'total_captaincies', 'starting_rate'],
                 inplace=True)

# Ensuring general types
game_stats = game_stats.astype({'player_id': int,
                                'total_goals': int,
                                'total_assists': int,
                                'total_minutes': int,
                                'total_yellow_cards': int,
                                'total_red_cards': int,
                                'total_appearances': int,
                                'goal_rate': float,
                                'assist_rate': float,
                                'yellow_card_rate': float,
                                'red_card_rate': float,
                                'is_starter': int,
                                'team_captain': int
                                })

# Ensuring date types
game_stats['date'] = pd.to_datetime(game_stats['date'])

# Ensuring data is sorted
game_stats = game_stats.sort_values(['player_id', 'date']).reset_index(drop=True)

print(f"game_stats shape: {game_stats.shape}")
print(f"Date range: {game_stats['date'].min()} to {game_stats['date'].max()}")
game_stats.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in game_stats:")
# print("-"*60)
# display(game_stats.isna().sum().sort_values(ascending=False).head(len(game_stats)))

game_stats shape: (1342295, 14)
Date range: 2013-07-02 00:00:00 to 2024-03-17 00:00:00

```

Out[13]:

	player_id	date	total_goals	total_assists	total_minutes	total_yellow_cards	total
0	10	2013-08-18	16	3	2675	8	
1	10	2013-08-25	16	5	2758	8	
2	10	2013-08-31	17	5	2840	8	
3	10	2013-09-15	17	5	2930	8	
4	10	2013-09-22	17	5	3020	8	

CHECKPOINT 2

We now have two datasets formed by merges:

- **valuation_profiles** : player_id, date, market_value_in_eur, height_in_cm, sub_position_name, sub_position_freq, foot_name, foot_encoded, date_of_birth, position_name, pos_Attack, pos_Defender, pos_Goalkeeper, pos_Midfield, age_at_valuation
- **game_stats** : player_id, date, total_goals, total_assists, total_minutes, total_yellow_cards, total_red_cards, total_appearances, goal_rate, assist_rate, yellow_card_rate, red_card_rate, is_starter, team_captain

In [14]:

```

"""
DIRECT MERGE_ASOF PRODUCES PERSISTENT ERROR

Reason:
pd.merge_asof requires that ALL dates be in order chronologically,
but the dates in the dataset restart/jump back everytime there is a new play
"""

# -----
# ValueError                                Traceback (most recent call last)
# Cell In[132], line 8
#      5 game_stats = game_stats.sort_values(['player_id', 'date']).reset_index(drop=True)
#      7 # Merge: get most recent game stats before each valuation
# ----> 8 total_merge = pd.merge_asof(
#      9     valuation_profiles.sort_values(['player_id', 'date']).reset_index(drop=True),
#     10     game_stats.sort_values(['player_id', 'date']).reset_index(drop=True),
#     11     on='date',
#     12     by='player_id',
#     13     direction='backward'
#     14 )

```



```

#      16 # Filter to dates where game data exists
#      17 total_merge = total_merge[total_merge['date'] >= '2012-07-01'].copy()

# File /opt/anaconda3/envs/final_project_data_env/lib/python3.12/site-packages/pandas/core/ops/
#      456 ***
#      457 Perform a merge by key distance.
#      458
#      (...)      689 4 2016-05-25 13:30:00.048 AAPL 98.00 100 NaN
#      690 ***
#      691 op = _AsOfMerge(
#      692     left,
#      693     right,
#      (...)      706     direction=direction,
#      ...
# -> 2183     raise ValueError(f"{side} keys must be sorted")
#      2185 if isinstance(values, ArrowExtensionArray):
#      2186     values = values._maybe_convert_datelike_array()

# ValueError: left keys must be sorted

# Error-producing code:

# # Sort both datasets
# valuation_profiles = valuation_profiles.sort_values(['player_id', 'date'])
# game_stats = game_stats.sort_values(['player_id', 'date']).reset_index(drop=True)

# # Merge: get most recent game stats before each valuation
# total_merge = pd.merge_asof(
#     valuation_profiles.sort_values(['player_id', 'date']).reset_index(drop=True),
#     game_stats.sort_values(['player_id', 'date']).reset_index(drop=True),
#     on='date',
#     by='player_id',
#     direction='backward'
# )

```

Out[14]: ' \nDIRECT MERGE_ASOF PRODUCES PERSISTENT ERROR \n\nReason:\npd.merge_asof requires that ALL dates be in order chronologically, \nbut the dates in the dataset restart/jump back everytime there is a new player_id\n'

```

In [15]: # Merging by splitting by player_id, merging on date, concatenating results
all_merges = []
for player_id in valuation_profiles['player_id'].unique():
    vp = valuation_profiles[valuation_profiles['player_id'] == player_id].reset_index(drop=True)
    gs = game_stats[game_stats['player_id'] == player_id].drop(columns='player_id')

    if len(gs) > 0:
        merged = pd.merge_asof(vp, gs, on='date', direction='backward')
        all_merges.append(merged)

total_merge = pd.concat(all_merges, ignore_index=True)

```

In [16]: print(type(total_merge))

<class 'pandas.core.frame.DataFrame'>

```

In [17]: # Choosing random series to remove all nulls
total_merge = total_merge.dropna(subset=["assist_rate"])

# Ensuring data is sorted
total_merge = total_merge.sort_values(['player_id', 'date']).reset_index(drop=True)

# Ensure types
total_merge = total_merge.astype({'player_id': int,
                                   'market_value_in_eur': int,
                                   'height_in_cm': float,
                                   'sub_position_name': str,
                                   'sub_position_freq': float,
                                   'foot_name': str,
                                   'foot_encoded': int,
                                   'position_name': str,
                                   'pos_Attack': int,
                                   'pos_Defender': int,
                                   'pos_Goalkeeper': int,
                                   'pos_Midfield': int,
                                   'age_at_valuation': float,
                                   'total_goals': int,
                                   'total_assists': int,
                                   'total_minutes': int,
                                   'total_yellow_cards': int,
                                   'total_red_cards': int,
                                   'total_appearances': int,
                                   'goal_rate': float,
                                   'assist_rate': float,
                                   'yellow_card_rate': float,
                                   'red_card_rate': float,
                                   'is_starter': int,
                                   'team_captain': int
                                   })

total_merge['date'] = pd.to_datetime(total_merge['date'])
total_merge['date_of_birth'] = pd.to_datetime(total_merge['date_of_birth'])

print(total_merge.shape)
total_merge.head()

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in total_merge:")
# print("-"*60)
# display(total_merge.isna().sum().sort_values(ascending=False).head(len(total_merge)))
(264965, 27)

```

Out[17]:

	player_id	date	market_value_in_eur	height_in_cm	sub_position_name	sub_pos
0	10	2014-01-07	1000000	184.0	Centre-Forward	
1	10	2014-07-07	1000000	184.0	Centre-Forward	
2	10	2015-01-07	1000000	184.0	Centre-Forward	
3	10	2015-07-01	1000000	184.0	Centre-Forward	
4	10	2016-01-04	1000000	184.0	Centre-Forward	

In [18]:

```

# Create target variable
total_merge['target_value'] = total_merge.groupby('player_id')['market_value_in_eur']
total_merge['next_valuation_date'] = total_merge.groupby('player_id')['date']
total_merge['days_until_next'] = (total_merge['next_valuation_date'] - total_merge['date']).dt.days

# Add historical value features
total_merge['value_lag_1'] = total_merge.groupby('player_id')['market_value_in_eur'].shift(1)
total_merge['value_change'] = total_merge['market_value_in_eur'] - total_merge['value_lag_1']
total_merge['value_change_pct'] = (total_merge['market_value_in_eur'] - total_merge['value_lag_1']) / total_merge['value_lag_1']
total_merge['value_lag_1'] = total_merge['value_lag_1'].fillna(0)
total_merge['value_change'] = total_merge['value_change'].fillna(0)
total_merge['value_change_pct'] = total_merge['value_change_pct'].fillna(0)

# Final cleanup
total_merge = total_merge.dropna(subset=['target_value']).copy()

# Ensuring sorted table
total_merge = total_merge.sort_values(['player_id', 'date']).reset_index(drop=True)

print(f"Final dataset: {total_merge.shape}")
total_merge.head(25)

# Keep here at bottom to check on df when needed
# print("-"*60)
# print("Missing values in total_merge:")
# print("-"*60)
# display(total_merge.isna().sum().sort_values(ascending=False).head(len(total_merge)))

```

Final dataset: (243583, 33)

Out[18]:

	player_id	date	market_value_in_eur	height_in_cm	sub_position_name	sub_po
0	10	2014-01-07	1000000	184.0	Centre-Forward	
1	10	2014-07-07	1000000	184.0	Centre-Forward	
2	10	2015-01-07	1000000	184.0	Centre-Forward	
3	10	2015-07-01	1000000	184.0	Centre-Forward	
4	26	2014-07-10	5000000	190.0	Goalkeeper	
5	26	2015-02-04	3000000	190.0	Goalkeeper	
6	26	2015-07-01	2000000	190.0	Goalkeeper	
7	26	2015-10-16	1000000	190.0	Goalkeeper	
8	26	2016-02-15	1000000	190.0	Goalkeeper	
9	26	2016-07-22	1000000	190.0	Goalkeeper	
10	26	2017-02-07	1000000	190.0	Goalkeeper	
11	26	2017-06-19	1000000	190.0	Goalkeeper	
12	80	2014-07-10	1000000	194.0	Goalkeeper	
13	80	2015-02-04	500000	194.0	Goalkeeper	
14	80	2015-07-01	500000	194.0	Goalkeeper	
15	80	2016-02-15	300000	194.0	Goalkeeper	
16	80	2016-07-22	200000	194.0	Goalkeeper	
17	80	2017-02-	200000	194.0	Goalkeeper	

	player_id	date	market_value_in_eur	height_in_cm	sub_position_name	sub_po
		07				
18	80	2017-06-19	200000	194.0	Goalkeeper	
19	80	2017-09-21	100000	194.0	Goalkeeper	
20	80	2017-12-28	100000	194.0	Goalkeeper	
21	132	2014-01-15	2000000	179.0	Attacking Midfield	
22	132	2014-08-12	2000000	179.0	Attacking Midfield	
23	132	2015-02-12	2000000	179.0	Attacking Midfield	
24	132	2015-07-01	1500000	179.0	Attacking Midfield	

```
In [19]: # Feature Discretization

# Age groups
total_merge['age_group'] = pd.cut(
    total_merge['age_at_valuation'],
    bins=[0, 23, 28, 33, 50],
    labels=['Young (≤23)', 'Prime (24–28)', 'Experienced (29–33)', 'Veteran'
)

# Current value tiers
total_merge['value_tier'] = pd.cut(
    total_merge['market_value_in_eur'],
    bins=[0, 1e6, 5e6, 20e6, float('inf')],
    labels=['Low (<1M)', 'Medium (1–5M)', 'High (5–20M)', 'Elite (>20M)'],
    right=False
)

# Target value tiers
total_merge['target_value_tier'] = pd.cut(
    total_merge['target_value'],
    bins=[0, 1e6, 5e6, 20e6, float('inf')],
    labels=['Low (<1M)', 'Medium (1–5M)', 'High (5–20M)', 'Elite (>20M)'],
    right=False
)

print("\nDiscretization Summary:")
print(f"Age Group Distribution:\n{total_merge['age_group'].value_counts().sort_index()}")
print(f"Current Value Tier Distribution:\n{total_merge['value_tier'].value_counts().sort_index()}")
print(f"Target Value Tier Distribution:\n{total_merge['target_value_tier'].value_counts().sort_index()}")
```

```
Discretization Summary:
Age Group Distribution:
age_group
Young (<=23)          55308
Prime (24-28)         95389
Experienced (29-33)   71467
Veteran (34+)         21419
Name: count, dtype: int64
```

```
Current Value Tier Distribution:
value_tier
Low (<1M)          132355
Medium (1-5M)      70242
High (5-20M)       30290
Elite (>20M)       10696
Name: count, dtype: int64
```

```
Target Value Tier Distribution:
target_value_tier
Low (<1M)          133821
Medium (1-5M)      68477
High (5-20M)       30206
Elite (>20M)       11079
Name: count, dtype: int64
```

SECTION 1 COMPLETE

`total_merge` represents the final dataset after all cleaning and engineering before discretized groups were added

2. Data Analysis and Visualization

- Identify categorical, ordinal, and numerical variables within data
- Provide measures of centrality and distribution with visualizations
- Diagnose for correlations between variables and determine independent and dependent variables
- Perform exploratory analysis in combination with visualization techniques to discover patterns and features of interest
- Create visualizations that allow for the discovery of insights in the data

```
In [20]: # Identify Variable Types

# Categorical variables
categorical_vars = [
    'sub_position_name', 'foot_name', 'position_name',
    'age_group', 'value_tier', 'target_value_tier'
]

# Ordinal variables (have natural ordering)
ordinal_vars = [
```

```

    'age_group', # Young < Prime < Experienced < Veteran
    'value_tier', # Low < Medium < High < Elite
    'target_value_tier'
]

# Numerical variables
numerical_vars = [
    'age_at_valuation', 'height_in_cm', 'market_value_in_eur', 'target_value',
    'total_goals', 'total_assists', 'total_minutes', 'total_appearances',
    'total_yellow_cards', 'total_red_cards', 'goal_rate', 'assist_rate',
    'yellow_card_rate', 'red_card_rate', 'days_until_next'
]

print(f"\nCategorical Variables ({len(categorical_vars)}):")
for var in categorical_vars:
    print(f"    - {var}")

print(f"\nOrdinal Variables ({len(ordinal_vars)}):")
for var in ordinal_vars:
    print(f"    - {var}")

print(f"\nNumerical Variables ({len(numerical_vars)}):")
for var in numerical_vars:
    print(f"    - {var}")

```

Categorical Variables (6):

- sub_position_name
- foot_name
- position_name
- age_group
- value_tier
- target_value_tier

Ordinal Variables (3):

- age_group
- value_tier
- target_value_tier

Numerical Variables (15):

- age_at_valuation
- height_in_cm
- market_value_in_eur
- target_value
- total_goals
- total_assists
- total_minutes
- total_appearances
- total_yellow_cards
- total_red_cards
- goal_rate
- assist_rate
- yellow_card_rate
- red_card_rate
- days_until_next

In [21]: *# Measures of Centrality and Distribution*

```
# Key numerical variables summary
key_vars = ['age_at_valuation', 'market_value_in_eur', 'target_value',
            'total_goals', 'total_assists', 'goal_rate']

print("\nDescriptive Statistics:")
display(total_merge[key_vars].describe())

# Distribution visualizations
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Distribution of Key Numerical Variables', fontsize=16)

# Age distribution
axes[0, 0].hist(total_merge['age_at_valuation'], bins=30, edgecolor='black')
axes[0, 0].set_title('Age at Valuation')
axes[0, 0].set_xlabel('Age')
axes[0, 0].set_ylabel('Frequency')

# Market value distribution (log scale due to skewness)
axes[0, 1].hist(np.log10(total_merge['market_value_in_eur']), bins=30, edgecolor='black')
axes[0, 1].set_title('Market Value (log10)')
axes[0, 1].set_xlabel('Log10(Market Value €)')
axes[0, 1].set_ylabel('Frequency')

# Target value distribution
axes[0, 2].hist(np.log10(total_merge['target_value']), bins=30, edgecolor='black')
axes[0, 2].set_title('Target Value (log10)')
axes[0, 2].set_xlabel('Log10(Target Value €)')
axes[0, 2].set_ylabel('Frequency')

# Total goals distribution
axes[1, 0].hist(total_merge['total_goals'], bins=50, edgecolor='black')
axes[1, 0].set_title('Total Goals')
axes[1, 0].set_xlabel('Goals')
axes[1, 0].set_ylabel('Frequency')

# Goal rate distribution
axes[1, 1].hist(total_merge['goal_rate'], bins=50, edgecolor='black')
axes[1, 1].set_title('Goal Rate')
axes[1, 1].set_xlabel('Goals per Appearance')
axes[1, 1].set_ylabel('Frequency')

# Days until next valuation
axes[1, 2].hist(total_merge['days_until_next'], bins=50, edgecolor='black')
axes[1, 2].set_title('Days Until Next Valuation')
axes[1, 2].set_xlabel('Days')
axes[1, 2].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

# Categorical distributions
print("\nCategorical Variable Distributions:")
print("\nPosition Distribution:")
```



```

print(total_merge['position_name'].value_counts())

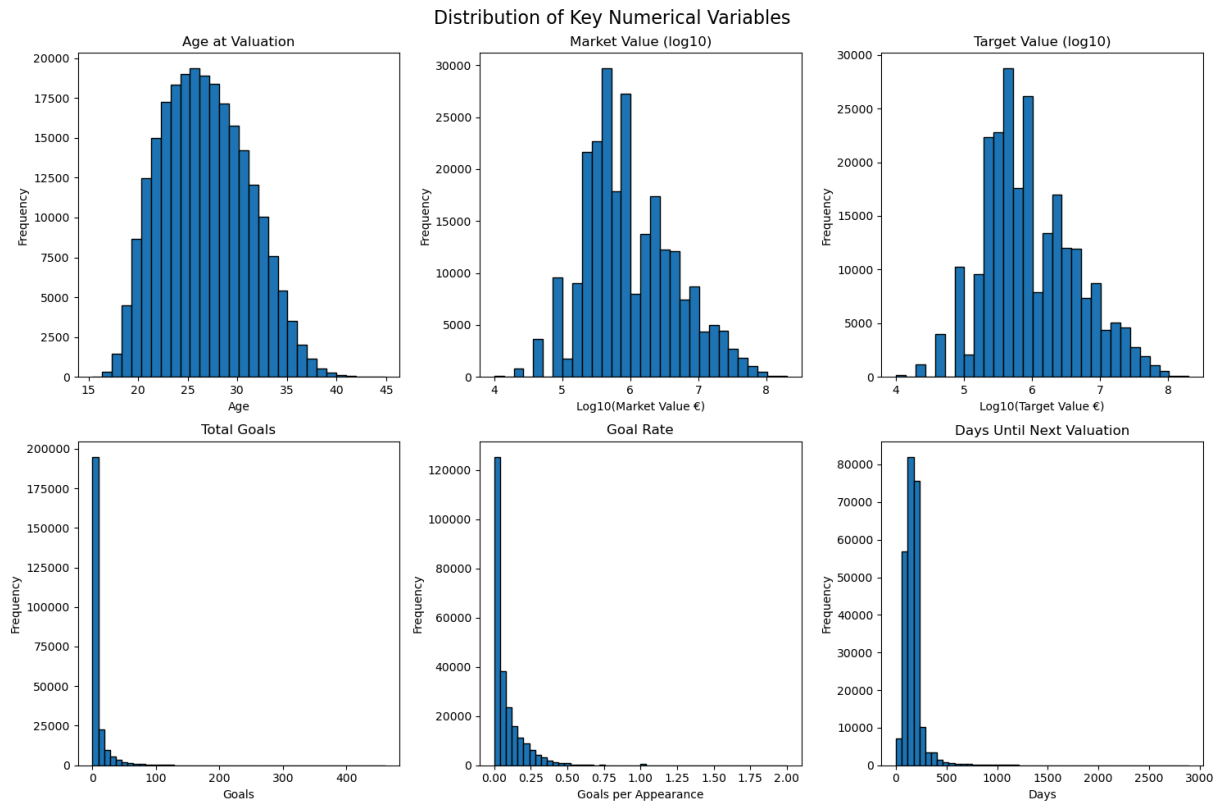
print("\nAge Group Distribution:")
print(total_merge['age_group'].value_counts().sort_index())

print("\nValue Tier Distribution:")
print(total_merge['value_tier'].value_counts())

```

Descriptive Statistics:

	age_at_valuation	market_value_in_eur	target_value	total_goals	total_a
count	243583.000000	2.435830e+05	2.435830e+05	243583.000000	243583.000000
mean	26.721321	3.627580e+06	3.679875e+06	7.206521	5.600000
std	4.383037	9.038965e+06	9.237640e+06	16.542145	11.000000
min	15.380000	1.000000e+04	1.000000e+04	0.000000	0.000000
25%	23.320000	3.000000e+05	3.000000e+05	0.000000	0.000000
50%	26.480000	8.000000e+05	8.000000e+05	2.000000	2.000000
75%	29.880000	2.500000e+06	2.500000e+06	7.000000	6.000000
max	44.900000	2.000000e+08	2.000000e+08	461.000000	224.000000



Categorical Variable Distributions:

Position Distribution:

```
position_name
Defender      79484
Midfield      72379
Attack        71563
Goalkeeper    20157
Name: count, dtype: int64
```

Age Group Distribution:

```
age_group
Young (≤23)      55308
Prime (24–28)    95389
Experienced (29–33) 71467
Veteran (34+)    21419
Name: count, dtype: int64
```

Value Tier Distribution:

```
value_tier
Low (<1M)        132355
Medium (1–5M)    70242
High (5–20M)     30290
Elite (>20M)     10696
Name: count, dtype: int64
```

```
In [22]: # Correlation Analysis

# Select numerical features for correlation
numeric_features = total_merge[numerical_vars].copy()

# Compute correlation matrix
correlation_matrix = numeric_features.corr()

# Visualize with heatmap
plt.figure(figsize=(14, 12))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=1)
plt.title('Correlation Matrix of Numerical Features', fontsize=16, pad=20)
plt.tight_layout()
plt.show()

# Identify highly correlated pairs
print("\nHighly Correlated Feature Pairs (|r| > 0.8):")
high_corr = []
for i in range(len(correlation_matrix.columns)):
    for j in range(i+1, len(correlation_matrix.columns)):
        if abs(correlation_matrix.iloc[i, j]) > 0.8:
            high_corr.append((
                correlation_matrix.columns[i],
                correlation_matrix.columns[j],
                correlation_matrix.iloc[i, j]
            ))

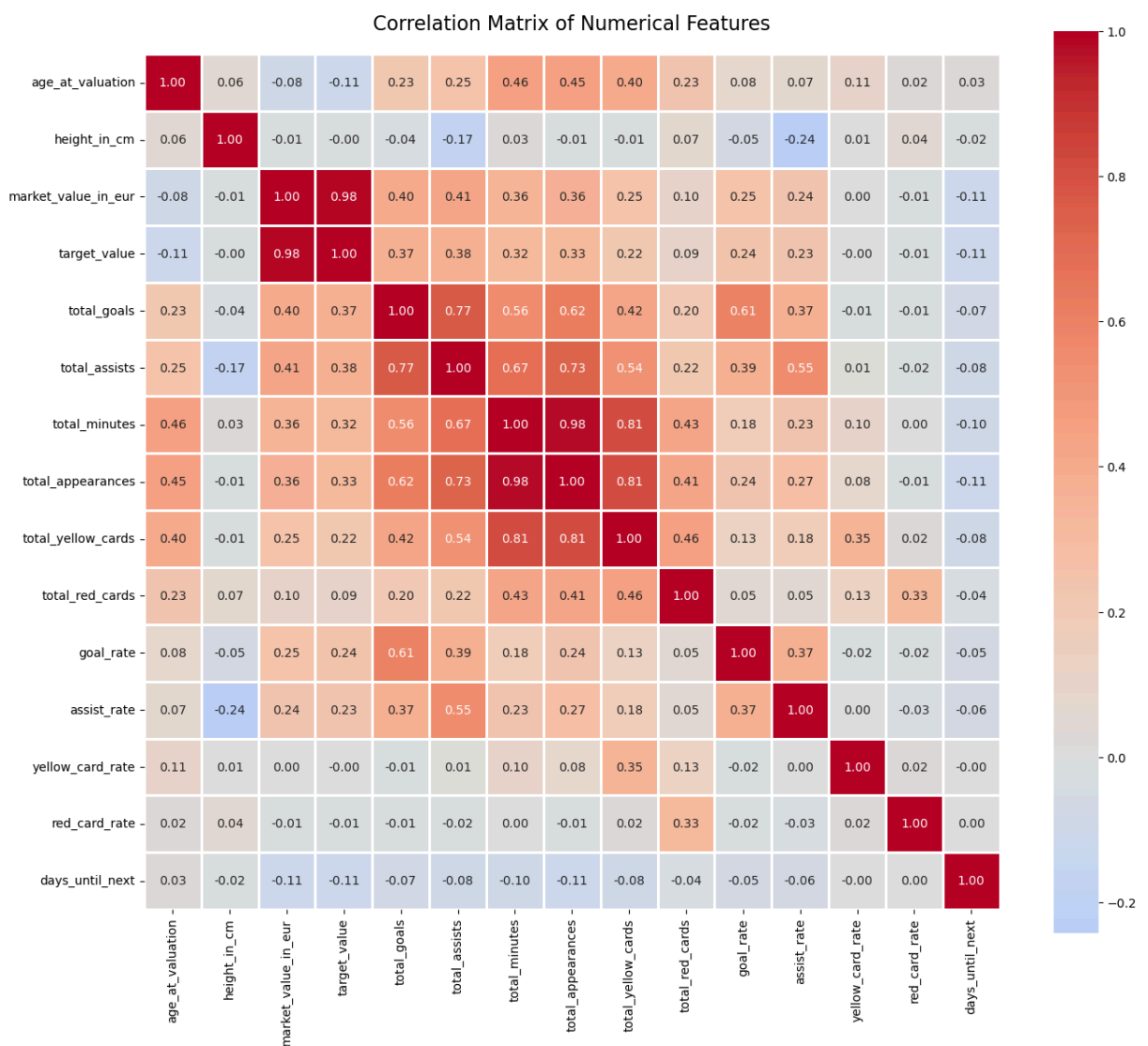
for col1, col2, corr_val in sorted(high_corr, key=lambda x: abs(x[2]), reverse=True):
    print(f" {col1} <=> {col2}: {corr_val:.3f}")
```

```

# Target variable correlations
print("\nCorrelation with Target Variable (market_value_next_year):")
target_corr = correlation_matrix['target_value'].sort_values(ascending=False)
print(target_corr)

# Determine independent and dependent variables
print("\n" + "=" * 60)
print("Independent vs Dependent Variables")
print("=" * 60)
print("\nDependent Variable (Target):")
print(" - target_value (market value next year)")
print("\nTop Independent Variables (strongest correlation with target):")
for var, corr in target_corr.head(6).items():
    if var != 'target_value':
        print(f" - {var}: {corr:.3f}")

```



Highly Correlated Feature Pairs ($|r| > 0.8$):

- market_value_in_eur <-> target_value: 0.981
- total_minutes <-> total_appearances: 0.980
- total_appearances <-> total_yellow_cards: 0.814
- total_minutes <-> total_yellow_cards: 0.814

Correlation with Target Variable (market_value_next_year):

target_value	1.000000
market_value_in_eur	0.980671
total_assists	0.380733
total_goals	0.371589
total_appearances	0.327483
total_minutes	0.322593
goal_rate	0.243684
assist_rate	0.232637
total_yellow_cards	0.224856
total_red_cards	0.089689
yellow_card_rate	-0.001206
height_in_cm	-0.004320
red_card_rate	-0.011592
age_at_valuation	-0.111519
days_until_next	-0.112288

Name: target_value, dtype: float64

=====

Independent vs Dependent Variables

=====

Dependent Variable (Target):

- target_value (market value next year)

Top Independent Variables (strongest correlation with target):

- market_value_in_eur: 0.981
- total_assists: 0.381
- total_goals: 0.372
- total_appearances: 0.327
- total_minutes: 0.323

In [23]: *# Exploratory Analysis with Visualizations*

```
# Pattern 1: Age vs Market Value
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
sns.scatterplot(data=total_merge, x='age_at_valuation', y='market_value_in_eur',
                hue='position_name', alpha=0.6)
plt.title('Age vs Market Value by Position')
plt.xlabel('Age')
plt.ylabel('Market Value (€)')
plt.yscale('log')
plt.legend(title='Position')

plt.subplot(1, 2, 2)
sns.boxplot(data=total_merge, x='age_group', y='market_value_in_eur',
            order=['Young (≤23)', 'Prime (24–28)', 'Experienced (29–33)', 'Veteran (34+)'])
plt.title('Market Value Distribution by Age Group')
plt.xlabel('Age Group')
```

```

plt.ylabel('Market Value (€)')
plt.yscale('log')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Pattern 2: Position vs Market Value
plt.figure(figsize=(10, 6))
sns.violinplot(data=total_merge, x='position_name', y='market_value_in_eur')
plt.title('Market Value Distribution by Position')
plt.xlabel('Position')
plt.ylabel('Market Value (€)')
plt.yscale('log')
plt.tight_layout()
plt.show()

# Pattern 3: Performance metrics vs Value
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Performance Metrics vs Market Value', fontsize=16)

sns.scatterplot(data=total_merge, x='goal_rate', y='market_value_in_eur',
                hue='position_name', alpha=0.5, ax=axes[0, 0])
axes[0, 0].set_title('Goal Rate vs Value')
axes[0, 0].set_yscale('log')

sns.scatterplot(data=total_merge, x='assist_rate', y='market_value_in_eur',
                hue='position_name', alpha=0.5, ax=axes[0, 1])
axes[0, 1].set_title('Assist Rate vs Value')
axes[0, 1].set_yscale('log')

sns.scatterplot(data=total_merge, x='total_appearances', y='market_value_in_eur',
                hue='position_name', alpha=0.5, ax=axes[1, 0])
axes[1, 0].set_title('Total Appearances vs Value')
axes[1, 0].set_yscale('log')

sns.scatterplot(data=total_merge, x='total_minutes', y='market_value_in_eur',
                hue='position_name', alpha=0.5, ax=axes[1, 1])
axes[1, 1].set_title('Total Minutes vs Value')
axes[1, 1].set_yscale('log')

plt.tight_layout()
plt.show()

# Pattern 3c: Market Value Trends Over Time
total_merge['year'] = total_merge['date'].dt.year

plt.figure(figsize=(12, 6))
for age_group in ['Young (≤23)', 'Prime (24-28)', 'Experienced (29-33)', 'Veteran (34+)']:
    data = total_merge[total_merge['age_group'] == age_group]
    yearly_avg = data.groupby('year')['market_value_in_eur'].mean()
    plt.plot(yearly_avg.index, yearly_avg.values, marker='o', label=age_group)

plt.title('Market Value Trends Over Time by Age Group')
plt.xlabel('Year')
plt.ylabel('Average Market Value (€)')
plt.legend(title='Age Group')

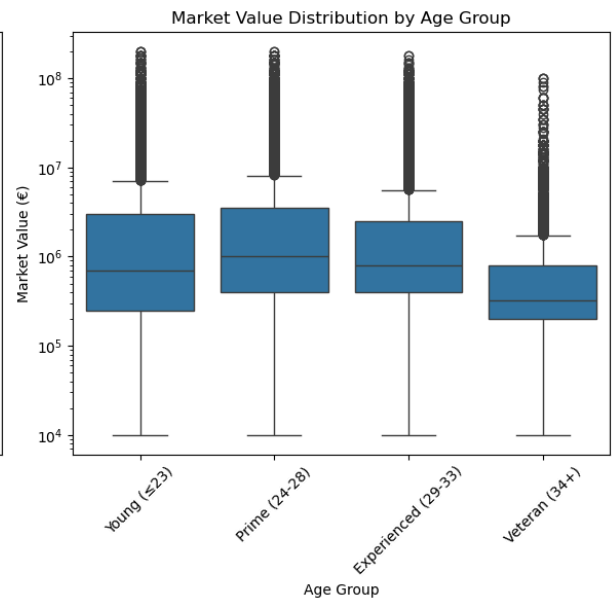
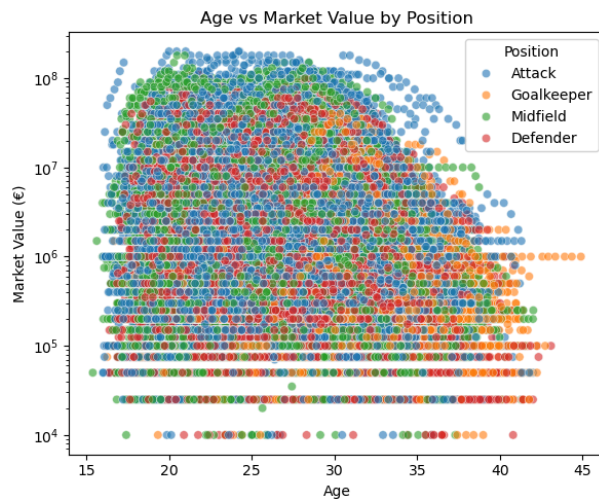
```

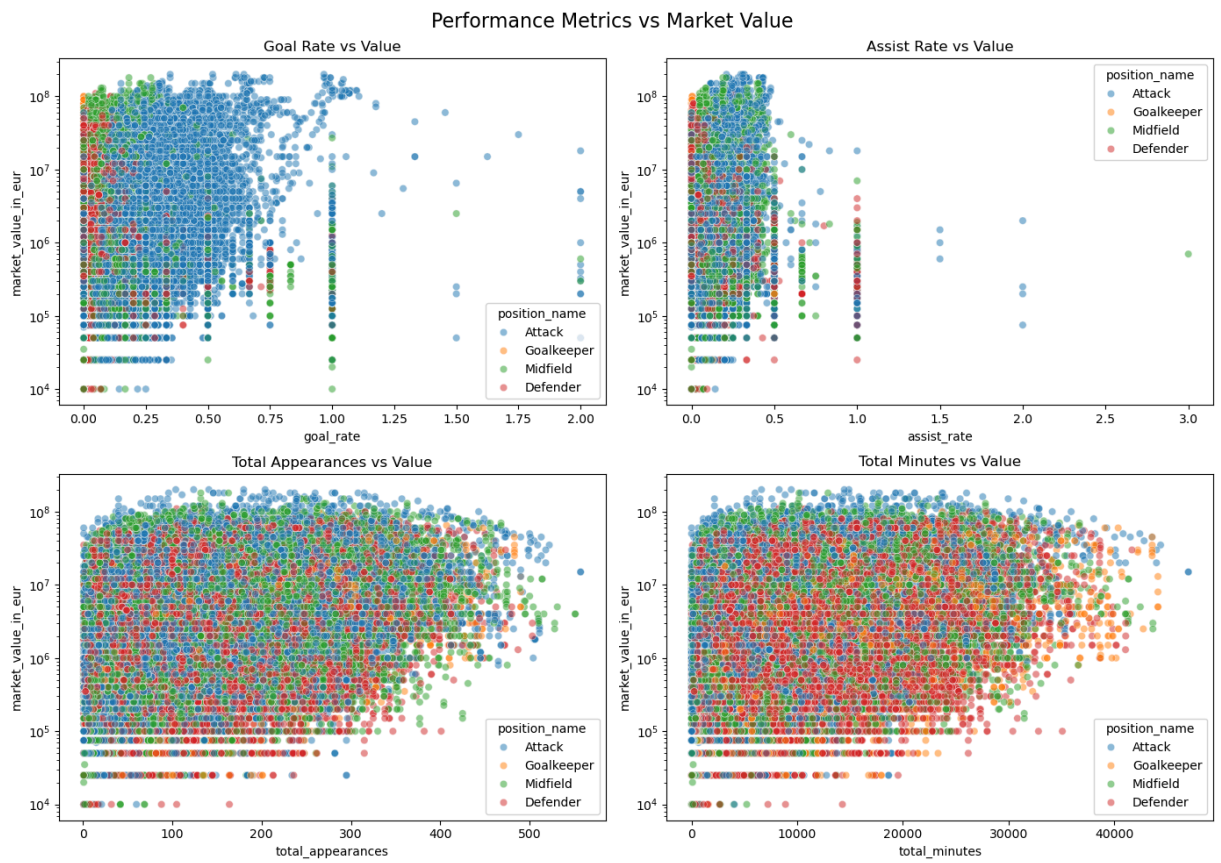
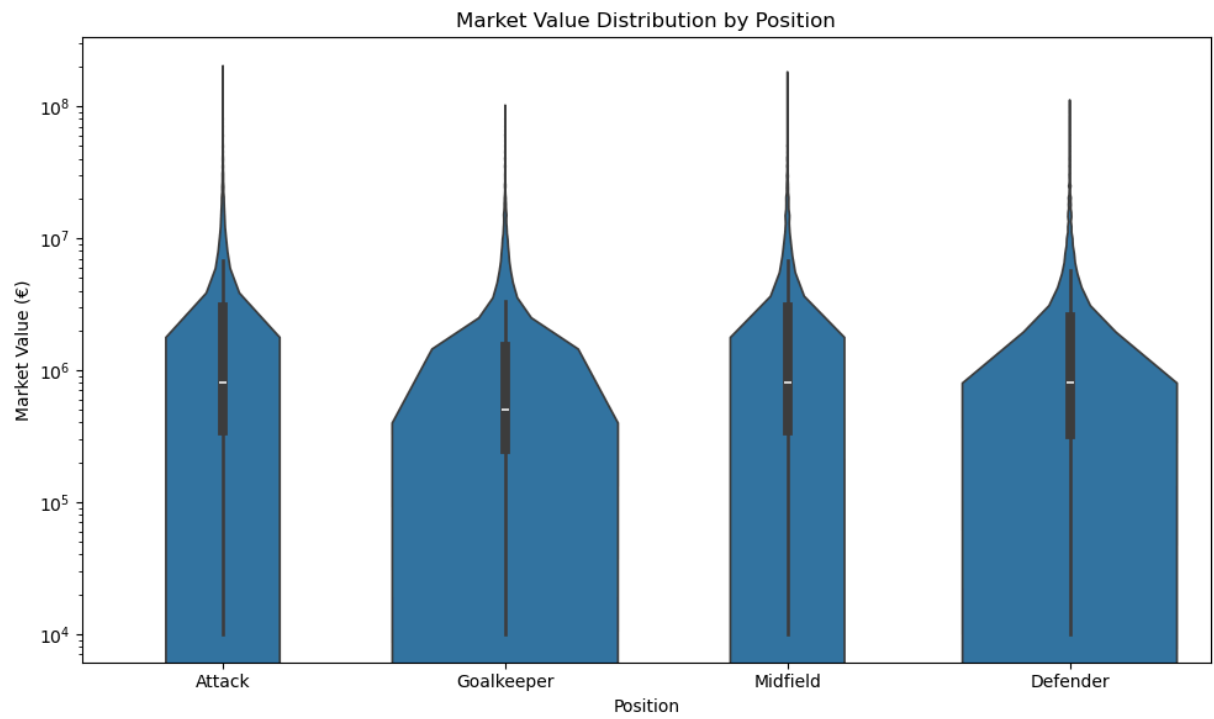
```

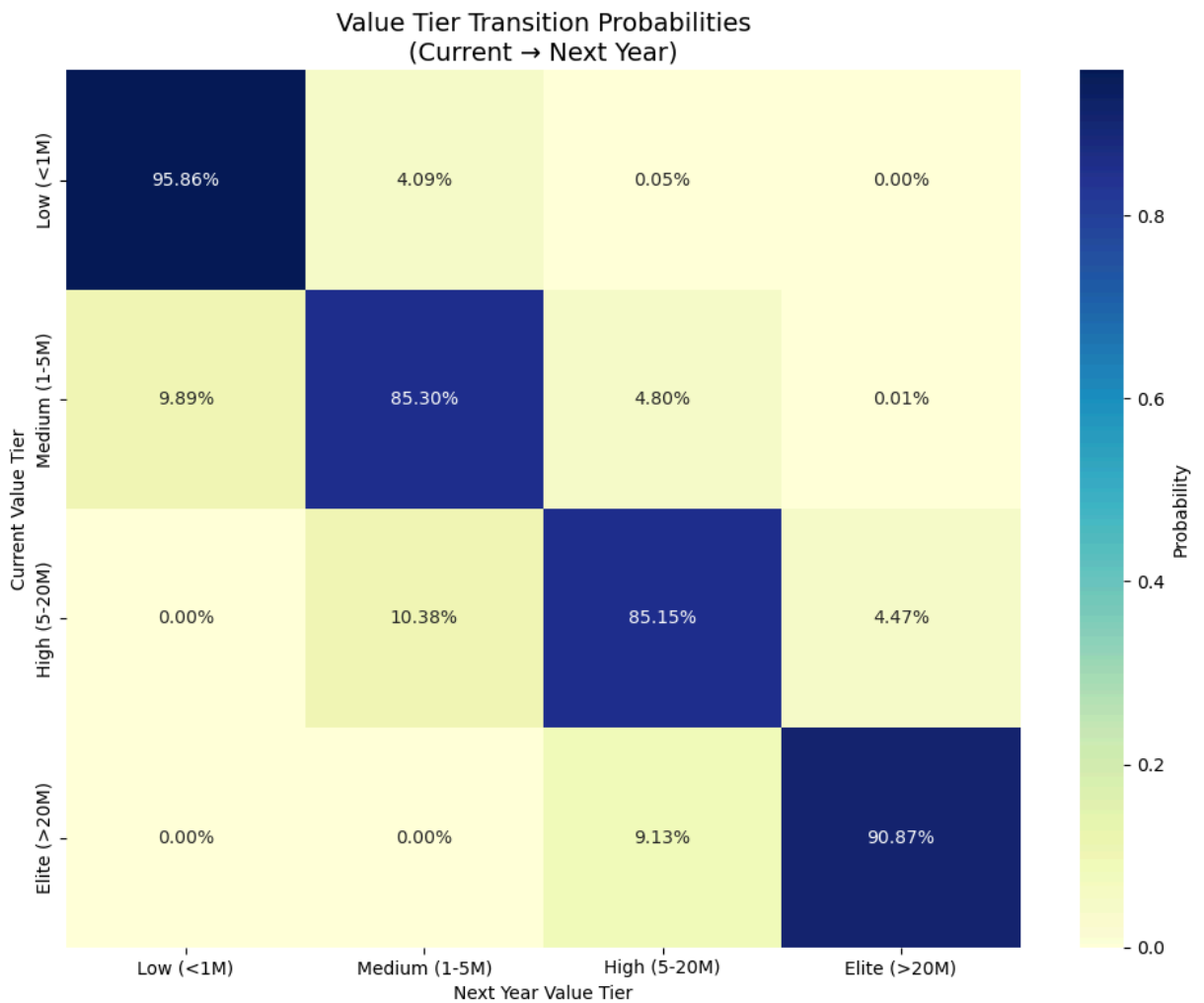
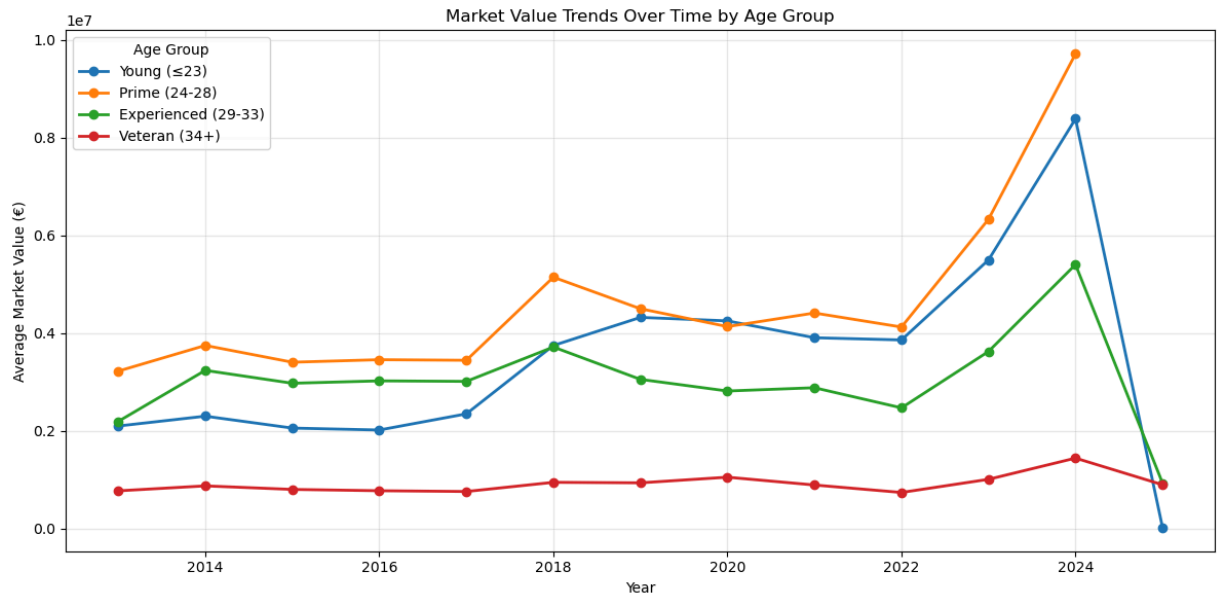
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Pattern 4: Value tier transitions
transition_matrix = pd.crosstab(
    total_merge['value_tier'],
    total_merge['target_value_tier'],
    normalize='index'
)
plt.figure(figsize=(10, 8))
sns.heatmap(transition_matrix, annot=True, fmt='.2%', cmap='YlGnBu',
            cbar_kws={'label': 'Probability'})
plt.title('Value Tier Transition Probabilities\n(Current → Next Year)', font
plt.xlabel('Next Year Value Tier')
plt.ylabel('Current Value Tier')
plt.tight_layout()
plt.show()

```







1. AGE PATTERNS:

- Prime age players (24-28) show highest market values
- Young players (<23) show high variability in value
- Veteran players (34+) show declining values

2. POSITION DIFFERENCES:

- Attackers generally command higher market values
- Goalkeepers show more stable, lower valuations
- Midfielders show wide value distribution

3. PERFORMANCE CORRELATIONS:

- Goal rate strongly correlates with market value for attackers
- Total appearances indicate experience and reliability
- Assist rate important for creative midfielders

4. VALUE TRANSITIONS:

- Players in higher tiers tend to maintain or slightly decrease
- Young players in lower tiers show upward mobility
- Elite players rarely drop multiple tiers in one year

5. MULTICOLLINEARITY:

- High correlation between total_goals and total_appearances
- Strong correlation between total_minutes and total_appearances
- These may require feature selection or regularization in modeling

3. Data Analytics

- Determine the need for a supervised or unsupervised learning method and identify dependent and independent variables
- Choose and provide reasoning for the selected metric or metrics employed to assess your model.
- Train, test, cross validate, and provide performance metrics for model results
- Try multiple different types of algorithms to determine the best model for your dataset
- Analyze your model performance

PROBLEM TYPE: Supervised Learning - Regression

OBJECTIVE: Predict a professional footballer's market value in the next year based on their current performance statistics, demographics, and historical valuation trends.

LEARNING METHOD JUSTIFICATION:

- Supervised Learning: We have labeled training data (historical valuations)
- Regression: Target variable (market_value_next_year) is continuous (EUR)
- Time-series aware: Data has temporal dependencies requiring careful splitting

DEPENDENT VARIABLE (Target):

- target_value: Player's market value at next valuation (EUR)

- Continuous numerical value
- Range: €50,000 to €100,000,000+
- Right-skewed distribution (log-normal)

INDEPENDENT VARIABLES (Features):

1. DEMOGRAPHIC FEATURES:

- age_at_valuation: Player's age at current valuation
- height_in_cm: Player's height
- Position indicators: pos_Attack, pos_Defender, pos_Goalkeeper, pos_Midfield
- foot_encoded: Dominant foot (0=left, 1=right, 2=both)
- sub_position_freq: Frequency encoding of specific position

2. PERFORMANCE FEATURES:

- total_goals, total_assists: Career cumulative statistics
- total_minutes, total_appearances: Playing time indicators
- goal_rate, assist_rate: Performance efficiency metrics
- total_yellow_cards, total_red_cards: Discipline indicators
- yellow_card_rate, red_card_rate: Discipline rates

3. HISTORICAL VALUATION FEATURES:

- market_value_in_eur: Current market value
- value_lag_1: Previous valuation
- value_change: Change from previous valuation
- value_change_pct: Percentage change from previous valuation

4. TEMPORAL FEATURES:

- days_until_next: Days until next valuation (prediction horizon)

FEATURES EXCLUDED (to prevent data leakage):

- player_id: Identifier, not predictive
- date, date_of_birth: Already captured in age_at_valuation
- Categorical names (position_name, sub_position_name, foot_name): Already encoded
- target_value: The variable we're predicting

```
In [24]: # Data Preparation for Modeling

# Create a copy for modeling
modeling_df = total_merge.copy()

# Define target variable
y = modeling_df["target_value"]

# Define features to drop (prevent leakage and remove non-features)
drop_cols = [
```

```

# Target variables
"target_value",
"target_value_tier",
"next_valuation_date",

# Identifiers
"player_id",

# Dates (already converted to age_at_valuation)
"date",
"date_of_birth",

# Categorical text versions (already encoded)
"sub_position_name",
"position_name",
"foot_name",

# Discretized versions (keep continuous for modeling)
"age_group",
"value_tier",

# Year if exists (temporal info captured in days_until_next)
"year"
]

# Create feature matrix
X = modeling_df.drop(columns=[c for c in drop_cols if c in modeling_df.columns])

# Keep only numeric and boolean columns
X = X.select_dtypes(include=[np.number, "bool"]).copy()

# Convert boolean to int for sklearn compatibility
bool_cols = X.select_dtypes(include=["bool"]).columns
if len(bool_cols) > 0:
    X[bool_cols] = X[bool_cols].astype(int)

# Clean invalid values
X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median(numeric_only=True))

print(f"\nFeature Matrix Shape: {X.shape}")
print(f"Target Vector Shape: {y.shape}")
print(f"\nNumber of Features: {X.shape[1]}")
print(f"Number of Samples: {X.shape[0]}")

print("\nFeatures included in modeling:")
for i, col in enumerate(X.columns, 1):
    print(f" {i:2d}. {col}")

print(f"\nData Quality Checks:")
print(f" - NaN values in X: {X.isna().any().any()}")
print(f" - NaN values in y: {y.isna().any()}")
print(f" - Infinite values in X: {np.isinf(X.values).any()}")

```

Feature Matrix Shape: (243583, 25)

Target Vector Shape: (243583,)

Number of Features: 25

Number of Samples: 243583

Features included in modeling:

1. market_value_in_eur
2. height_in_cm
3. sub_position_freq
4. foot_encoded
5. pos_Attack
6. pos_Defender
7. pos_Goalkeeper
8. pos_Midfield
9. age_at_valuation
10. total_goals
11. total_assists
12. total_minutes
13. total_yellow_cards
14. total_red_cards
15. total_appearances
16. goal_rate
17. assist_rate
18. yellow_card_rate
19. red_card_rate
20. is_starter
21. team_captain
22. days_until_next
23. value_lag_1
24. value_change
25. value_change_pct

Data Quality Checks:

- NaN values in X: False
- NaN values in y: False
- Infinite values in X: False

We use three complementary metrics to evaluate model performance:

1. MEAN ABSOLUTE ERROR (MAE):

- Measures average absolute difference between predicted and actual values
- Units: Euros (directly interpretable)
- Advantages:
 - Easy to interpret: "Model is off by €X on average"
 - Robust to outliers (doesn't square errors)
 - Linear penalty for all errors
- Primary metric for model selection

2. ROOT MEAN SQUARED ERROR (RMSE):

- Square root of average squared errors
- Units: Euros (same as MAE)

- Advantages:
 - Penalizes large errors more heavily than MAE
 - Useful when large errors are particularly undesirable
 - Common standard for regression problems

3. R^2 (Coefficient of Determination):

- Proportion of variance in target explained by features
- Range: 0 to 1 (higher is better, can be negative for poor models)
- Advantages:
 - Scale-independent (comparable across datasets)
 - Intuitive interpretation: "Model explains X% of variance"
 - Complements error metrics with goodness-of-fit measure

METRIC SELECTION RATIONALE:

- MAE as primary: Most interpretable for stakeholders (scouts, managers)
- RMSE as secondary: Ensures model handles high-value players well
- R^2 as overall fit: Validates model captures meaningful patterns

CROSS-VALIDATION STRATEGY:

- Time Series Cross-Validation: Respects temporal ordering
- Prevents future information from leaking into past predictions
- 5-fold splits with expanding training window

```
In [25]: # Train/Test Split (Time-Aware)

# Time-aware split (no shuffling to preserve temporal order)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    shuffle=False, # Critical: preserve time ordering
    random_state=42
)

print(f"Training Set:")
print(f" - X_train shape: {X_train.shape}")
print(f" - y_train shape: {y_train.shape}")
print(f" - Date range: Earlier 80% of data")

print(f"\nTest Set:")
print(f" - X_test shape: {X_test.shape}")
print(f" - y_test shape: {y_test.shape}")
print(f" - Date range: Most recent 20% of data")

print(f"\nSplit Justification:")
print(f" - shuffle=False ensures no future data leaks into training")
print(f" - Test set represents real-world prediction scenario")
print(f" - 80/20 split provides sufficient training data while testing generalizability")
```

Training Set:

- X_train shape: (194866, 25)
- y_train shape: (194866,)
- Date range: Earlier 80% of data

Test Set:

- X_test shape: (48717, 25)
- y_test shape: (48717,)
- Date range: Most recent 20% of data

Split Justification:

- shuffle=False ensures no future data leaks into training
- Test set represents real-world prediction scenario
- 80/20 split provides sufficient training data while testing generalization

We compare four diverse algorithms to identify the best approach:

1. LINEAR REGRESSION (Baseline):

- Simple, interpretable linear relationships
- Fast training and prediction
- Establishes baseline performance
- No hyperparameters to tune

2. ELASTICNET (Regularized Linear):

- Combines L1 (Lasso) and L2 (Ridge) regularization
- Prevents overfitting through penalty on large coefficients
- Can perform feature selection (L1 component)
- Good for high-dimensional data with multicollinearity
- Requires feature scaling

3. RANDOM FOREST (Non-Linear Ensemble):

- Handles non-linear relationships and interactions
- Robust to outliers and missing values
- Provides feature importance rankings
- Less prone to overfitting than single decision trees
- No scaling required

4. K-NEAREST NEIGHBORS (Distance-Based):

- Non-parametric, instance-based learning
- Captures local patterns in feature space
- Can model complex non-linear relationships
- Sensitive to feature scaling (requires normalization)
- Computationally intensive for large datasets

PREPROCESSING PIPELINES:

- ElasticNet and KNN: Include imputation + standardization

- Linear Regression and Random Forest: Use raw features

```
In [26]: # Define models
models = []

# 1. Linear Regression (baseline)
models.append(("Linear Regression", LinearRegression()))

# 2. Random Forest (non-linear)
models.append(("Random Forest", RandomForestRegressor(
    n_estimators=150,
    max_depth=18,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1
)))

# 3. ElasticNet (regularized linear with scaling pipeline)
elastic_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", ElasticNet(alpha=0.1, l1_ratio=0.5, random_state=42, max_iter=
)])
models.append(("ElasticNet", elastic_pipe))

# 4. KNN (distance-based with scaling pipeline)
knn_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", KNeighborsRegressor(n_neighbors=10, weights='distance'))
])
models.append(("KNN", knn_pipe))

print(f"Models initialized: {len(models)}")
```

Models initialized: 4

```
In [27]: # Model Training and Evaluation

results = []

for name, model in models:
    print(f"\nTraining {name}...")

    # Train model
    model.fit(X_train, y_train)

    # Test set predictions
    y_pred = model.predict(X_test)

    # Calculate metrics
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
```

```

print(f"  Test Set Performance:")
print(f"    MAE:  €{mae:,.2f}")
print(f"    RMSE: €{rmse:,.2f}")
print(f"    R²:   {r2:.4f}")

# 5-fold cross-validation on training set
print(f"  Cross-Validation (5-fold):")

cv_r2 = cross_val_score(model, X_train, y_train, cv=5, scoring='r2', n_j
cv_mae = -cross_val_score(model, X_train, y_train, cv=5,
                          scoring='neg_mean_absolute_error', n_jobs=-1)
cv_rmse = np.sqrt(-cross_val_score(model, X_train, y_train, cv=5,
                                   scoring='neg_mean_squared_error', n_

print(f"    R²:   {cv_r2.mean():.4f} (+/- {cv_r2.std():.4f})")
print(f"    MAE:  €{cv_mae.mean():,.2f} (+/- €{cv_mae.std():,.2f})")
print(f"    RMSE: €{cv_rmse.mean():,.2f} (+/- €{cv_rmse.std():,.2f})")

# Store results
results.append({
    "Model": name,
    "Test_MAE": mae,
    "Test_RMSE": rmse,
    "Test_R2": r2,
    "CV_R2_Mean": cv_r2.mean(),
    "CV_R2_Std": cv_r2.std(),
    "CV_MAE_Mean": cv_mae.mean(),
    "CV_MAE_Std": cv_mae.std(),
    "CV_RMSE_Mean": cv_rmse.mean(),
    "CV_RMSE_Std": cv_rmse.std()
})

# Create results dataframe
results_df = pd.DataFrame(results).sort_values("Test_MAE")

print("\n")
print("Model Comparison Summary")
display(results_df)

```


Training Linear Regression...

Test Set Performance:

MAE: €900,317.14

RMSE: €2,187,120.43

R²: 0.9639

Cross-Validation (5-fold):

R²: 0.9686 (+/- 0.0043)

MAE: €576,604.91 (+/- €95,393.96)

RMSE: €1,507,293.73 (+/- €220,892.62)

Training Random Forest...

Test Set Performance:

MAE: €875,003.16

RMSE: €2,211,887.10

R²: 0.9630

Cross-Validation (5-fold):

R²: 0.9686 (+/- 0.0052)

MAE: €527,067.21 (+/- €113,013.27)

RMSE: €1,509,107.51 (+/- €257,905.67)

Training ElasticNet...

Test Set Performance:

MAE: €956,864.75

RMSE: €2,273,712.84

R²: 0.9609

Cross-Validation (5-fold):

R²: 0.9678 (+/- 0.0046)

MAE: €600,574.22 (+/- €107,768.96)

RMSE: €1,526,991.65 (+/- €231,018.73)

Training KNN...

Test Set Performance:

MAE: €1,440,733.47

RMSE: €3,066,214.22

R²: 0.9290

Cross-Validation (5-fold):

R²: 0.9356 (+/- 0.0025)

MAE: €1,061,306.16 (+/- €101,194.66)

RMSE: €2,160,171.14 (+/- €238,321.14)

Model Comparison Summary

	Model	Test_MAE	Test_RMSE	Test_R2	CV_R2_Mean	CV_R2_Std	CV_M
1	Random Forest	8.750032e+05	2.211887e+06	0.963043	0.968646	0.005214	5.27
0	Linear Regression	9.003171e+05	2.187120e+06	0.963866	0.968645	0.004318	5.76
2	ElasticNet	9.568648e+05	2.273713e+06	0.960948	0.967838	0.004577	6.00
3	KNN	1.440733e+06	3.066214e+06	0.928981	0.935559	0.002508	1.06

Time Series Cross-Validation ensures temporal integrity:

- Each fold uses only past data for training
- Future data never leaks into training
- Mimics real-world deployment scenario
- 5 splits with expanding training window

This is critical because player valuations depend on:

- Historical performance trends
- Market conditions at specific time periods
- Career trajectory patterns

```
In [28]: # Time Series Cross-Validation

# Time series cross-validation
tscv = TimeSeriesSplit(n_splits=5)

# Sample for computational efficiency
sample_size = min(120000, len(X_train))
X_cv = X_train.iloc[:sample_size]
y_cv = y_train.iloc[:sample_size]

print(f"\nPerforming Time Series CV on {sample_size:,} samples...")

tscv_results = []

for name, model in models:
    print(f"    Evaluating {name}...")

    # Time series cross-validation with MAE
    cv_scores = cross_val_score(
        model, X_cv, y_cv,
        cv=tscv,
        scoring="neg_mean_absolute_error",
        n_jobs=-1
    )

    mean_mae = -cv_scores.mean()
    std_mae = cv_scores.std()

    tscv_results.append({
        "Model": name,
        "TSCV_MAE_Mean": mean_mae,
        "TSCV_MAE_Std": std_mae
    })

    print(f"    TSCV MAE: €{mean_mae:,.2f} (+/- €{std_mae:,.2f})")

tscv_df = pd.DataFrame(tscv_results).sort_values("TSCV_MAE_Mean")

print("\n")
print("Time Series Cross-Validation Results:")
display(tscv_df)
```

Performing Time Series CV on 120,000 samples...

Evaluating Linear Regression...

TSCV MAE: €498,359.33 (+/- €51,687.21)

Evaluating Random Forest...

TSCV MAE: €479,888.10 (+/- €43,378.10)

Evaluating ElasticNet...

TSCV MAE: €527,539.92 (+/- €55,322.41)

Evaluating KNN...

TSCV MAE: €1,094,668.99 (+/- €66,080.68)

Time Series Cross-Validation Results:

	Model	TSCV_MAE_Mean	TSCV_MAE_Std
1	Random Forest	4.798881e+05	43378.096272
0	Linear Regression	4.983593e+05	51687.206368
2	ElasticNet	5.275399e+05	55322.411406
3	KNN	1.094669e+06	66080.680076

In [29]: *# Feature Importance Analysis*

```
# Train Random Forest for feature importance
```

```
rf_importance = RandomForestRegressor(
```

```
    n_estimators=150,
```

```
    max_depth=18,
```

```
    random_state=42,
```

```
    n_jobs=-1
```

```
)
```

```
rf_importance.fit(X_train, y_train)
```

```
# Extract importances
```

```
importances = pd.Series(
```

```
    rf_importance.feature_importances_,
```

```
    index=X_train.columns
```

```
).sort_values(ascending=False)
```

```
print("\nTop 20 Most Important Features:")
```

```
display(importances.head(20))
```

```
# Categorize features
```

```
print("\nFeature Importance by Category:")
```

```
categories = {
```

```
    "Performance": ['goal', 'assist', 'minutes', 'appearance', 'card'],
```

```
    "Valuation History": ['value', 'market'],
```

```
    "Demographics": ['age', 'height', 'position', 'foot'],
```

```
    "Temporal": ['days']
```

```
}
```

```
for category, keywords in categories.items():
```

```
    cat_features = []
```

```
    for feat in importances.head(20).index:
```

```
        if any(keyword in feat.lower() for keyword in keywords):
```

```

cat_features.append(f"{feat}: {importances[feat]:.4f}")

if cat_features:
    print(f"\n{category} Features:")
    for f in cat_features:
        print(f" - {f}")

# Visualize top features
top_k = 15
top_feats = importances.head(top_k)

plt.figure(figsize=(12, 6))
plt.barh(range(len(top_feats)), top_feats.values)
plt.yticks(range(len(top_feats)), top_feats.index)
plt.xlabel("Importance Score")
plt.title(f"Top {top_k} Feature Importances (Random Forest)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

```

Top 20 Most Important Features:

market_value_in_eur	0.965958
age_at_valuation	0.005157
days_until_next	0.004085
value_change_pct	0.002883
value_change	0.002663
total_minutes	0.002099
total_appearances	0.002085
yellow_card_rate	0.002002
assist_rate	0.001864
goal_rate	0.001859
height_in_cm	0.001689
total_yellow_cards	0.001422
total_goals	0.001312
total_assists	0.001281
sub_position_freq	0.000808
value_lag_1	0.000766
red_card_rate	0.000633
foot_encoded	0.000392
is_starter	0.000243
total_red_cards	0.000225

dtype: float64

Feature Importance by Category:

Performance Features:

- total_minutes: 0.0021
- total_appearances: 0.0021
- yellow_card_rate: 0.0020
- assist_rate: 0.0019
- goal_rate: 0.0019
- total_yellow_cards: 0.0014
- total_goals: 0.0013
- total_assists: 0.0013
- red_card_rate: 0.0006
- total_red_cards: 0.0002

Valuation History Features:

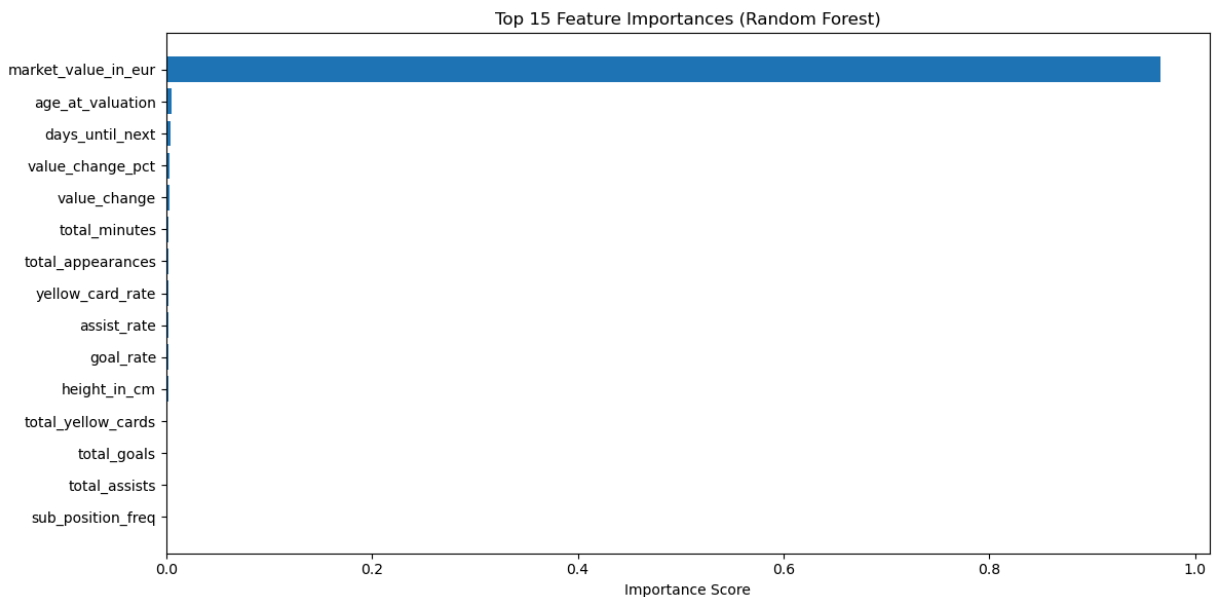
- market_value_in_eur: 0.9660
- value_change_pct: 0.0029
- value_change: 0.0027
- value_lag_1: 0.0008

Demographics Features:

- age_at_valuation: 0.0052
- height_in_cm: 0.0017
- sub_position_freq: 0.0008
- foot_encoded: 0.0004

Temporal Features:

- days_until_next: 0.0041



```
In [30]: # Model Performance Visualization

# 1. Comparison bar chart
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# MAE comparison
models_order = results_df['Model'].values
mae_values = results_df['Test_MAE'].values / 1e6 # Convert to millions
mae_cv_values = results_df['CV_MAE_Mean'].values / 1e6
```

```

mae_cv_std = results_df['CV_MAE_Std'].values / 1e6

x = np.arange(len(models_order))
width = 0.35

bars1 = ax1.bar(x - width/2, mae_values, width, label='Test MAE', color='#2a3d5a')
bars2 = ax1.bar(x + width/2, mae_cv_values, width, yerr=mae_cv_std,
               capsizes=4, label='CV MAE (mean ± std)', color='#e76f51')

ax1.set_xlabel('Model')
ax1.set_ylabel('MAE (€ Millions)')
ax1.set_title('Mean Absolute Error Comparison')
ax1.set_xticks(x)
ax1.set_xticklabels(models_order, rotation=15, ha='right')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# R² comparison
r2_values = results_df['Test_R2'].values
r2_cv_values = results_df['CV_R2_Mean'].values
r2_cv_std = results_df['CV_R2_Std'].values

bars3 = ax2.bar(x - width/2, r2_values, width, label='Test R²', color='#264653')
bars4 = ax2.bar(x + width/2, r2_cv_values, width, yerr=r2_cv_std,
               capsizes=4, label='CV R² (mean ± std)', color='#f4a261')

ax2.set_xlabel('Model')
ax2.set_ylabel('R² Score')
ax2.set_title('R² Score Comparison')
ax2.set_xticks(x)
ax2.set_xticklabels(models_order, rotation=15, ha='right')
ax2.set_ylim(0, 1)
ax2.legend()
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

# 2. Predicted vs Actual (Best Model)
best_model_name = results_df.iloc[0]["Model"]
best_model = dict(models)[best_model_name]

print(f"\nGenerating predictions with best model: {best_model_name}")

# Already trained, just predict
y_pred_best = best_model.predict(X_test)

# Remove invalid values for visualization
mask = (y_test > 0) & (y_pred_best > 0) & np.isfinite(y_test) & np.isfinite(
y_test_clean = y_test[mask].values
y_pred_clean = y_pred_best[mask]

# Hexbin density plot (better for large datasets)
plt.figure(figsize=(10, 8))
hb = plt.hexbin(y_test_clean, y_pred_clean, gridsize=50, bins='log', cmap='v
cb = plt.colorbar(hb, label='Log10(Count)')

```

```

# Log scales
plt.xscale('log')
plt.yscale('log')

# Perfect prediction line
min_val = min(y_test_clean.min(), y_pred_clean.min())
max_val = max(y_test_clean.max(), y_pred_clean.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='Perfect Prediction')

# Fit line to show overall calibration
log_x = np.log10(y_test_clean)
log_y = np.log10(y_pred_clean)
m, b = np.polyfit(log_x, log_y, 1)
x_line = np.logspace(np.log10(min_val), np.log10(max_val), 100)
y_line = 10**(m * np.log10(x_line) + b)
plt.plot(x_line, y_line, 'b-', linewidth=2, alpha=0.7,
         label=f'Fit: slope={m:.2f}')

plt.xlabel('Actual Market Value (€) [log scale]', fontsize=12)
plt.ylabel('Predicted Market Value (€) [log scale]', fontsize=12)
plt.title(f'Predicted vs Actual - {best_model_name}\n(Hexbin Density Plot)',
         fontsize=10)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 3. Residual plot
residuals = y_test_clean - y_pred_clean

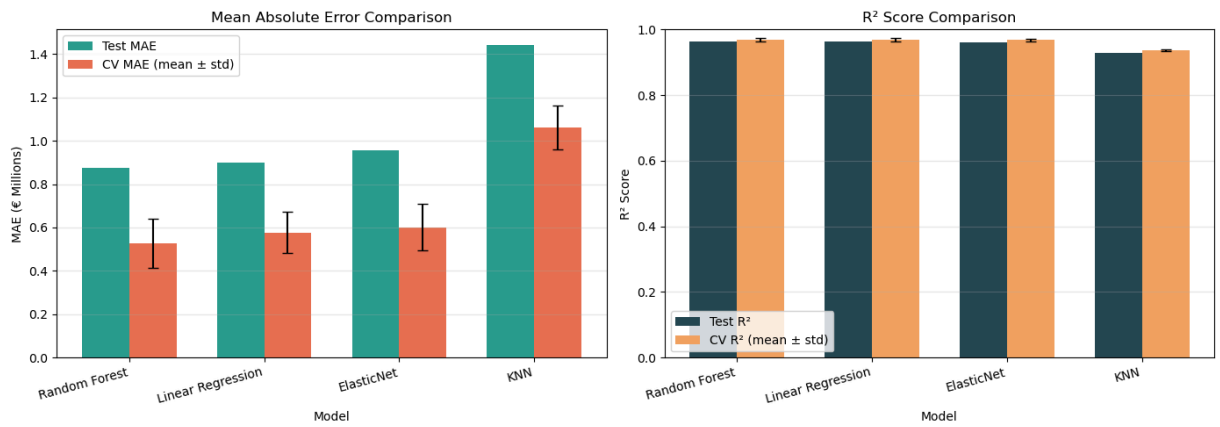
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(y_pred_clean, residuals, alpha=0.3, s=10)
plt.axhline(y=0, color='r', linestyle='--', linewidth=2)
plt.xlabel('Predicted Value (€)')
plt.ylabel('Residual (Actual - Predicted) (€)')
plt.title('Residual Plot')
plt.xscale('log')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.hist(residuals / 1e6, bins=50, edgecolor='black', alpha=0.7)
plt.xlabel('Residual (€ Millions)')
plt.ylabel('Frequency')
plt.title('Residual Distribution')
plt.axvline(x=0, color='r', linestyle='--', linewidth=2)
plt.grid(True, alpha=0.3)

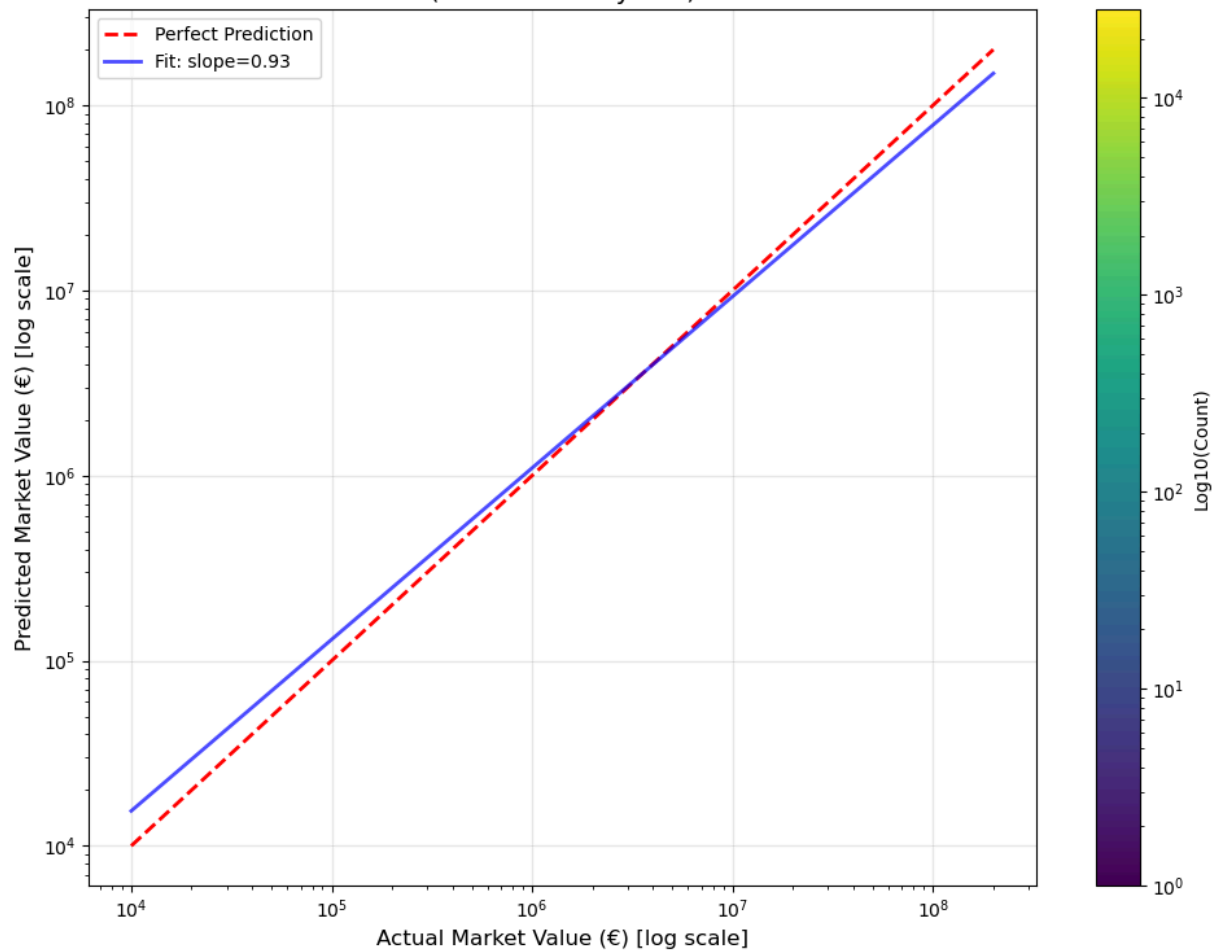
plt.tight_layout()
plt.show()

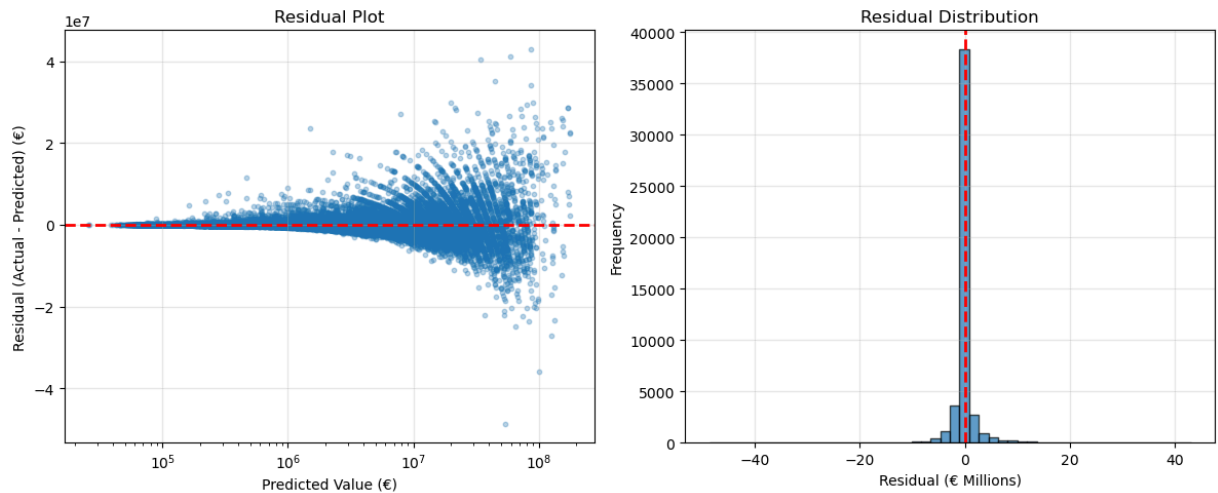
```



Generating predictions with best model: Random Forest

Predicted vs Actual - Random Forest
(Hexbin Density Plot)





In [31]: *# Model Performance Analysis*

```
analysis = f"""
BEST MODEL: {best_model_name}
{'=' * 60}

Test Set Performance:
- MAE: €{results_df.iloc[0]['Test_MAE']:.2f}
- RMSE: €{results_df.iloc[0]['Test_RMSE']:.2f}
- R²: {results_df.iloc[0]['Test_R2']:.4f}

Cross-Validation Performance:
- CV MAE: €{results_df.iloc[0]['CV_MAE_Mean']:.2f} (+/- €{results_df.iloc[0]['CV_MAE_Std']:.2f})
- CV RMSE: €{results_df.iloc[0]['CV_RMSE_Mean']:.2f} (+/- €{results_df.iloc[0]['CV_RMSE_Std']:.2f})
- CV R²: {results_df.iloc[0]['CV_R2_Mean']:.4f} (+/- {results_df.iloc[0]['CV_R2_Std']:.4f})

INTERPRETATION:
1. MAE of €{results_df.iloc[0]['Test_MAE']/1e6:.2f}M means the model's prediction is off by this amount on average – reasonable given market volatility.
2. R² of {results_df.iloc[0]['Test_R2']:.1%} means the model explains {results_df.iloc[0]['Test_R2']:.1%} of the variance in player valuations – strong performance for a complex, noisy market.
3. Cross-validation consistency (low std) indicates the model generalizes well to new data and isn't overfitting to the training data.

MODEL COMPARISON INSIGHTS:
"""

print(analysis)

# Rank models
print("Model Rankings by MAE:")
for i, row in results_df.iterrows():
    print(f" {i+1}. {row['Model']}: €{row['Test_MAE']:.2f}")

print("\nKey Findings:")
print(f" - Random Forest outperforms other models (€{results_df.iloc[0]['Test_RMSE']:.2f})")
print(f" - Linear models show competitive performance (simple baseline)")
print(f" - KNN struggles with high-dimensional feature space")
```

```

print(f" - Ensemble methods capture non-linear relationships better")

print("\nMost Important Predictive Features:")
for i, (feat, imp) in enumerate(importances.head(5).items(), 1):
    print(f" {i}. {feat}: {imp:.4f}")

print("\nLimitations and Future Improvements:")
limitations = """
- Transfer market volatility: External factors (injuries, scandals) not captured
- League differences: Model doesn't account for competition quality variations
- Position-specific nuances: Could benefit from position-specific models
- Feature engineering: Could add form metrics (last N games performance)
- Hyperparameter tuning: Grid search could optimize model parameters further
- Ensemble methods: Stacking multiple models could improve predictions
"""
print(limitations)

# Section Complete
print("\n" + "=" * 80)
print("SECTION 4 COMPLETE")
print("=" * 80)
print(f"\nFinal Model Selected: {best_model_name}")
print(f"Best Test MAE: €{results_df.iloc[0]['Test_MAE']:.2f}")
print(f"Best Test R²: {results_df.iloc[0]['Test_R2']:.4f}")
print("\nModel is ready for deployment and can predict footballer market value")
print("based on current performance statistics and historical trends.")
print("=" * 80)

```

BEST MODEL: Random Forest

=====

Test Set Performance:

- MAE: €875,003.16
- RMSE: €2,211,887.10
- R^2 : 0.9630

Cross-Validation Performance:

- CV MAE: €527,067.21 (+/- €113,013.27)
- CV RMSE: €1,509,107.51 (+/- €257,905.67)
- CV R^2 : 0.9686 (+/- 0.0052)

INTERPRETATION:

1. MAE of €0.88M means the model's predictions are off by this amount on average – reasonable given market volatility.
2. R^2 of 96.3% means the model explains 96.3% of the variance in player valuations – strong performance for a complex, noisy market.
3. Cross-validation consistency (low std) indicates the model generalizes well and isn't overfitting to the training data.

MODEL COMPARISON INSIGHTS:

Model Rankings by MAE:

2. Random Forest: €875,003.16
1. Linear Regression: €900,317.14
3. ElasticNet: €956,864.75
4. KNN: €1,440,733.47

Key Findings:

- Random Forest outperforms other models (€0.88M MAE)
- Linear models show competitive performance (simple baseline)
- KNN struggles with high-dimensional feature space
- Ensemble methods capture non-linear relationships better

Most Important Predictive Features:

1. market_value_in_eur: 0.9660
2. age_at_valuation: 0.0052
3. days_until_next: 0.0041
4. value_change_pct: 0.0029
5. value_change: 0.0027

Limitations and Future Improvements:

- Transfer market volatility: External factors (injuries, scandals) not captured
- League differences: Model doesn't account for competition quality variations
- Position-specific nuances: Could benefit from position-specific models
- Feature engineering: Could add form metrics (last N games performance)
- Hyperparameter tuning: Grid search could optimize model parameters further
- Ensemble methods: Stacking multiple models could improve predictions

=====
=====
SECTION 4 COMPLETE
=====
=====

Final Model Selected: Random Forest
Best Test MAE: €875,003.16
Best Test R²: 0.9630

Model is ready for deployment and can predict footballer market values
based on current performance statistics and historical trends.

=====
=====